第四章
基于共享内存的并行计算

哈尔滨工业大学
张伟哲

2024, Fall Semester

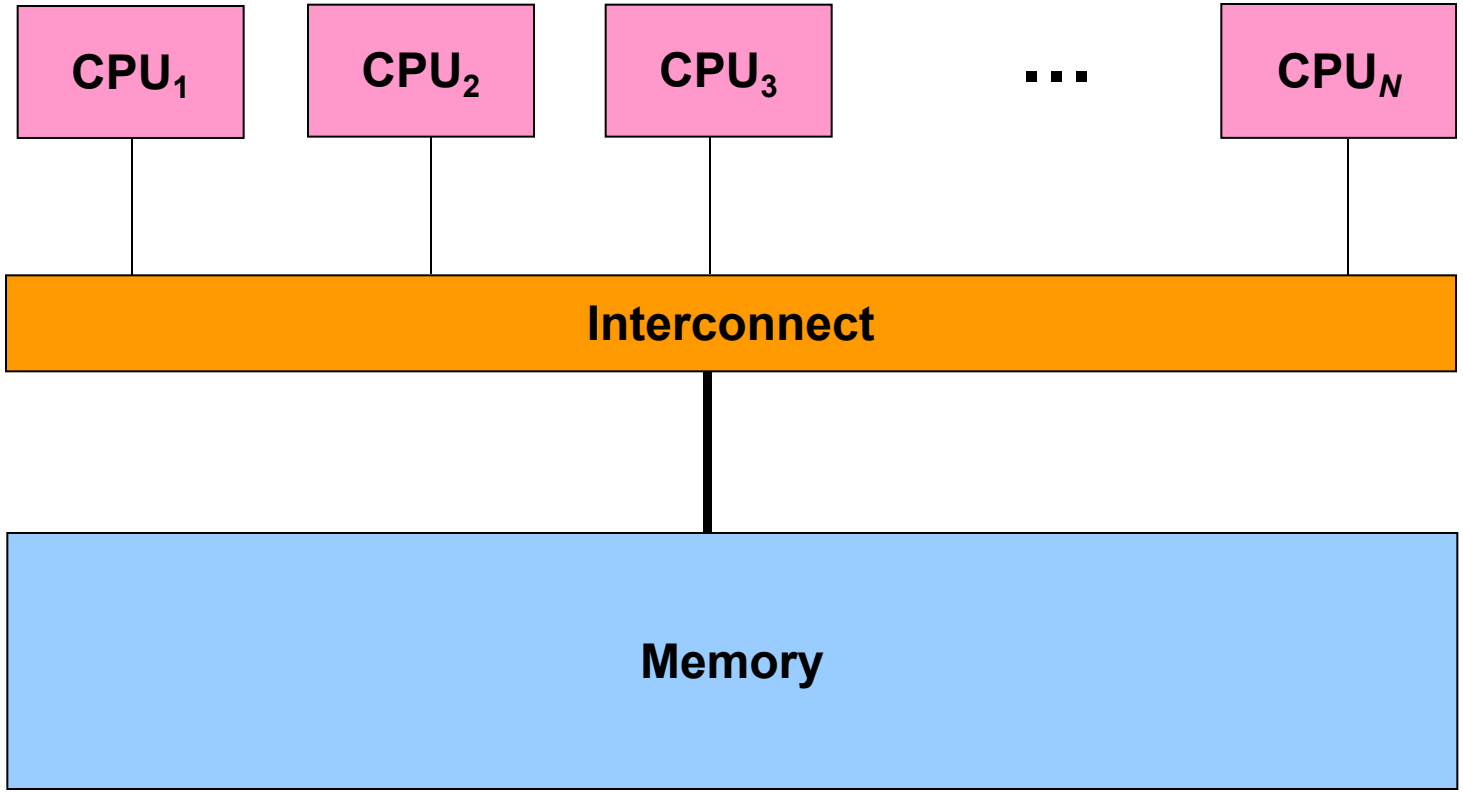
目录

- 进程与线程
- 基于Pthread的多线程并行
- 基于OpenMP的多线程并行

目录

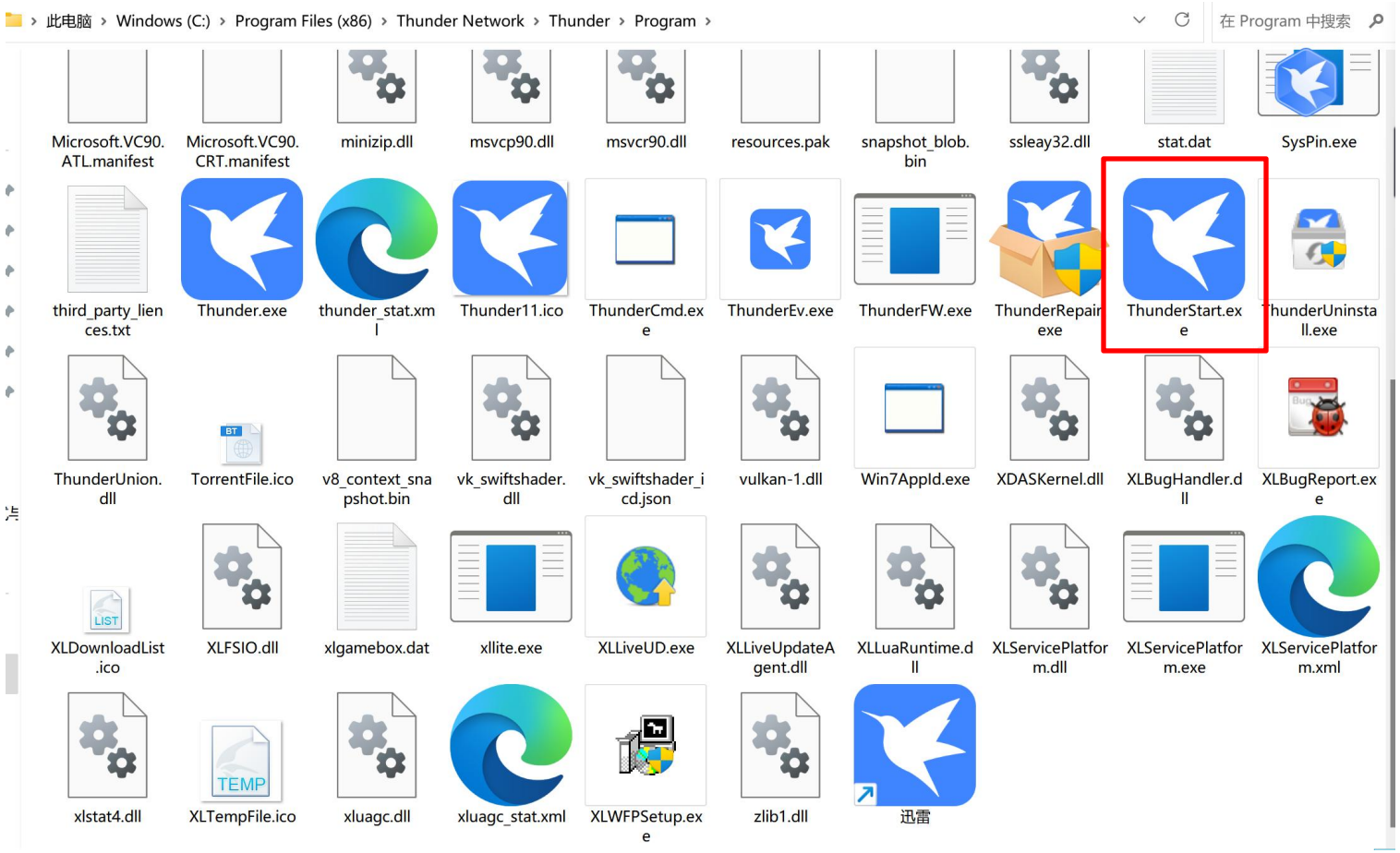
- 进程与线程
- 基于Pthread的多线程并行
- 基于OpenMP的多线程并行

共享内存系统



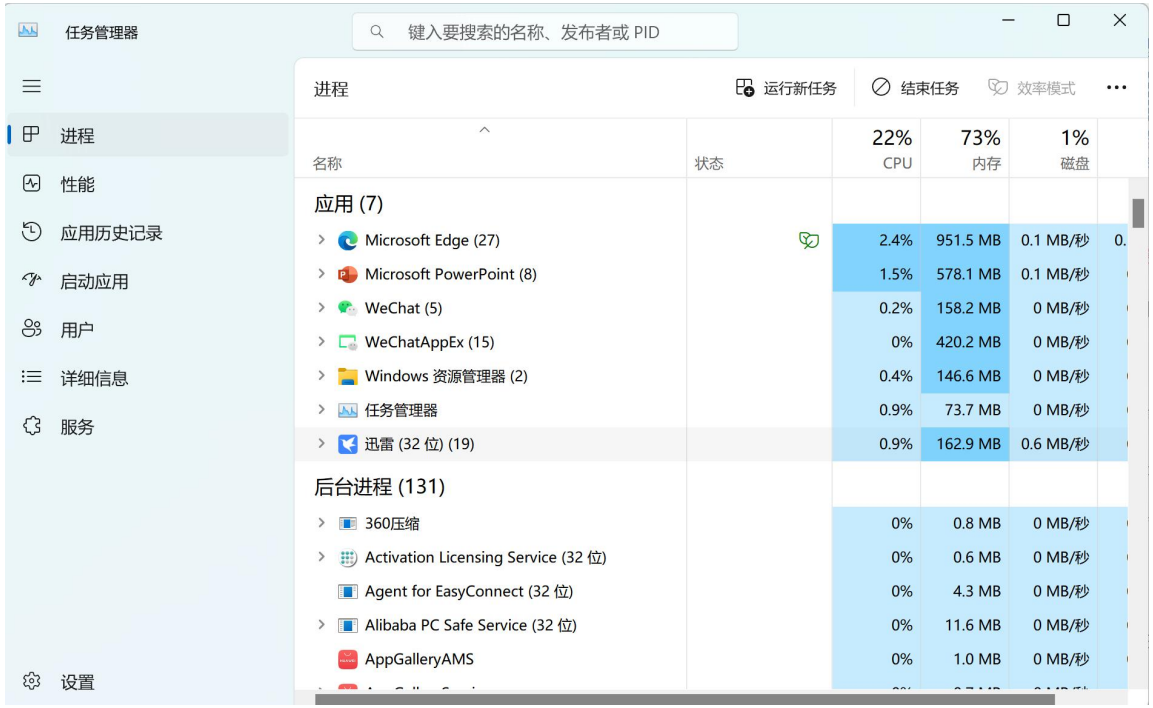
程序

- **程序是静态文件**

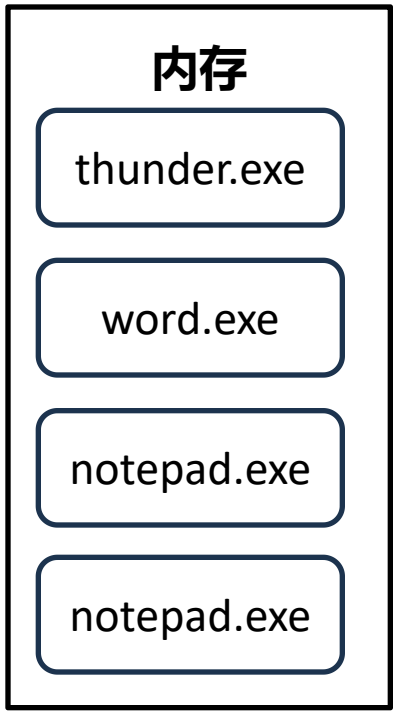

此电脑 › Windows (C:) › Program Files (x86) › Thunder Network › Thunder › Program ›
在 Program 中搜索
Microsoft.VC90.ATL.manifest
Microsoft.VC90.CRT.manifest
minizip.dll
msvcp90.dll
msvcr90.dll
resources.pak
snapshot_blob.bin
ssleay32.dll
stat.dat
SysPin.exe
third_party_licences.txt
Thunder.exe
thunder_stat.xml
Thunder11.ico
ThunderCmd.exe
ThunderEv.exe
ThunderFW.exe
ThunderRepair.exe
ThunderStart.exe
ThunderUninstall.exe
ThunderUnion.dll
TorrentFile.ico
v8_context_snapshot.bin
vk_swiftshader.dll
vk_swiftshader_icd.json
vulkan-1.dll
Win7AppId.exe
XDASKernel.dll
XLBugHandler.dll
XLBugReport.exe
XLDownloadList.ico
XLFSIO.dll
xlgamebox.dat
xllite.exe
XLLiveUD.exe
XLLiveUpdateAgent.dll
XLLuaRuntime.dll
XLServicePlatform.dll
XLServicePlatform.exe
XLServicePlatform.xml
xlstat4.dll
XLTempFile.ico
xluagc.dll
xluagc_stat.xml
XLWFPSetup.exe
zlib1.dll
迅雷

进程

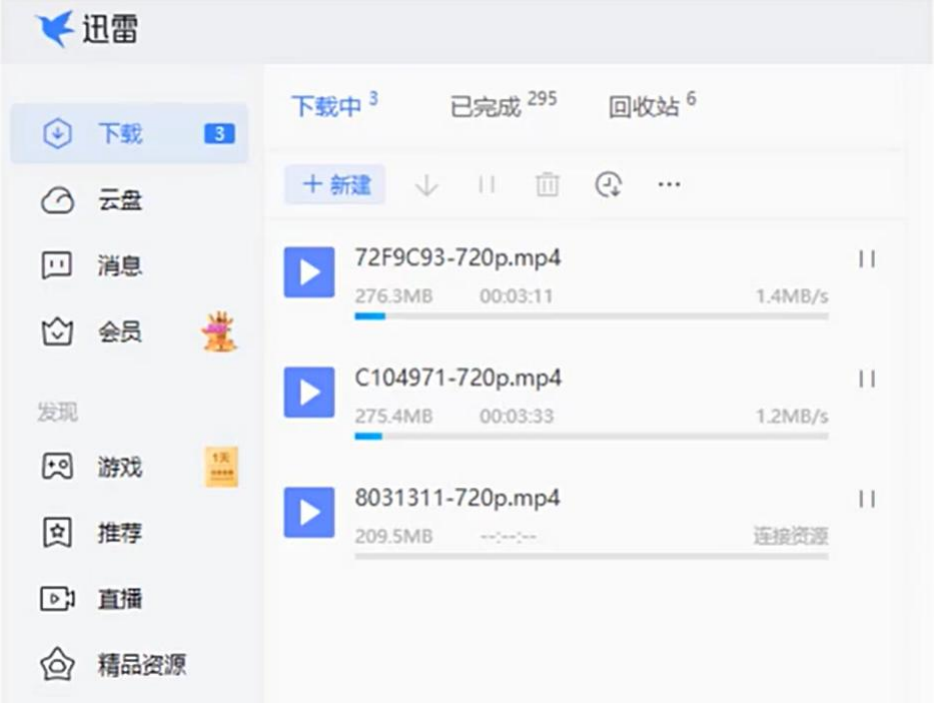
- **进程是程序运行的实例，当一个进程执行进入内存运行时，即变成一个进程**
- **进程的资源是彼此隔离的，其他进程不允许访问**



任务管理器



线程

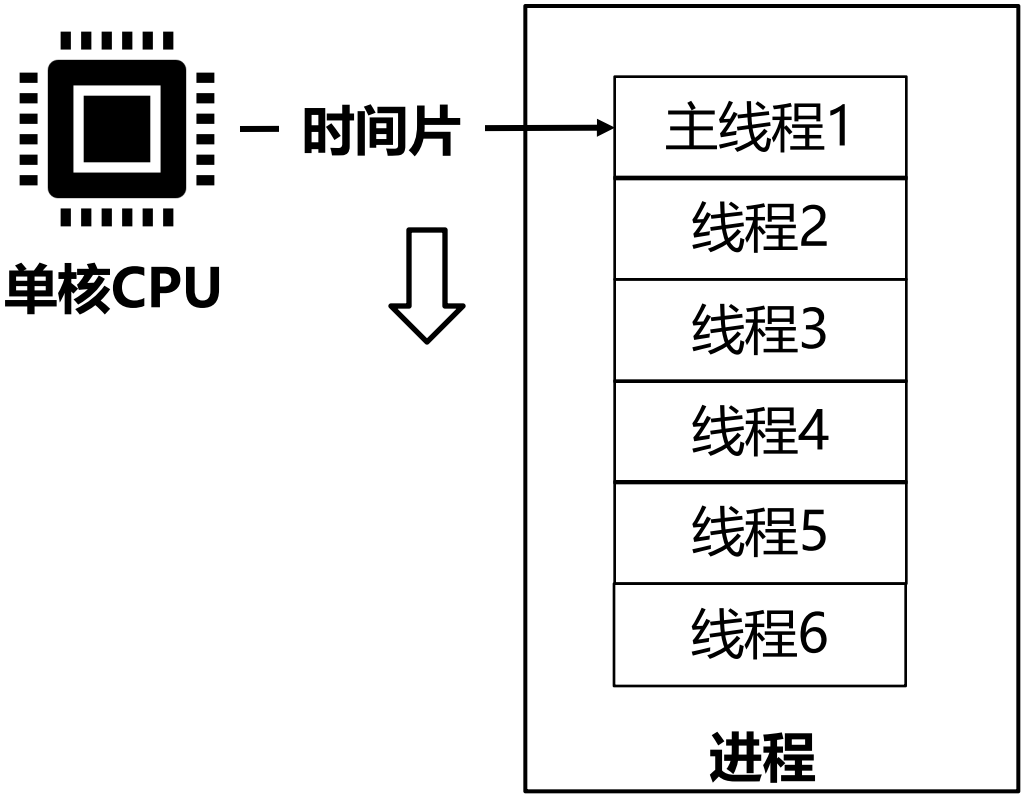

迅雷
下载
3
云盘
消息
会员
发现
游戏
推荐
直播
精品资源
下载中 3
已完成 295
回收站 6
+ 新建
72F9C93-720p.mp4
276.3MB
00:03:11
1.4MB/s
C104971-720p.mp4
275.4MB
00:03:33
1.2MB/s
8031311-720p.mp4
209.5MB
--:--:--
连接资源

主线程1
线程2
线程3
线程4
线程5
线程6
进程

线程

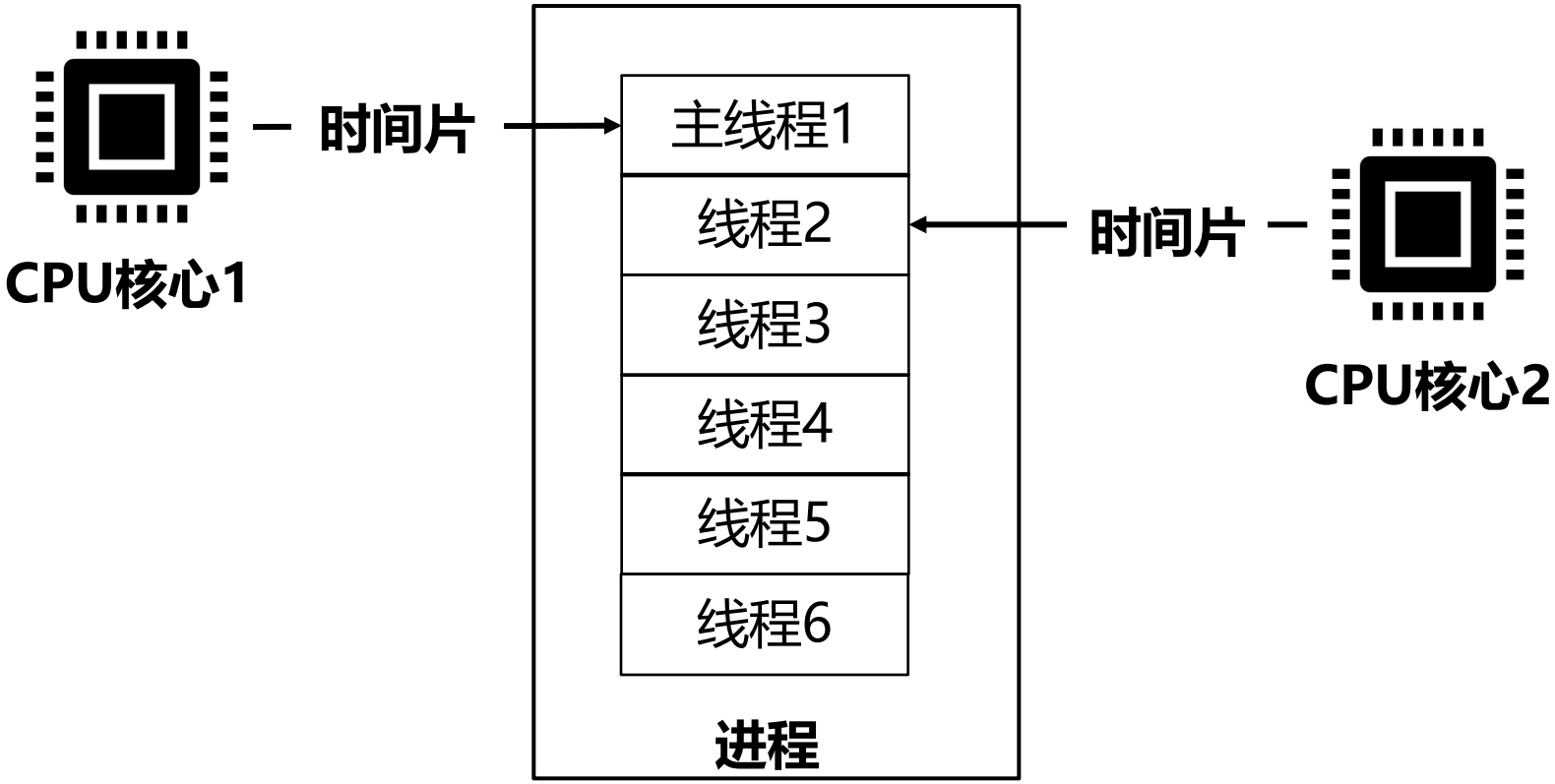
- 线程是进程执行的“基本任务”，每个线程都有自己的功能，是CPU分配与调度的基本单位
- 一个进程内可以包含多个线程，反之，一个线程只能隶属于某一个进程
- 进程内至少拥有一个“线程”，这个线程叫“主线程”，主线程消亡则进程结束
- 轻量级“进程”

CPU、进程、线程的关系



并发执行

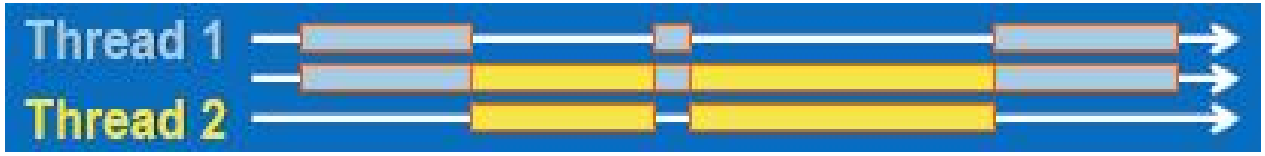
CPU、进程、线程的关系



并行执行

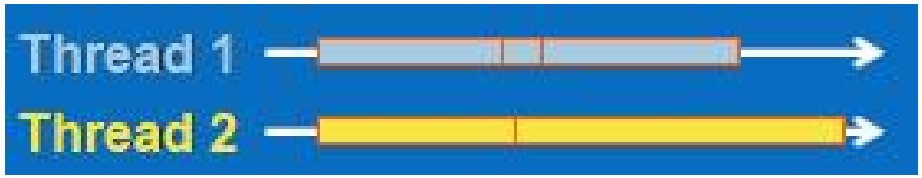
CPU、进程、线程的关系

- 单核平台，线程并发执行



CPU核

- 多核平台，各个核上可以实现线程并行

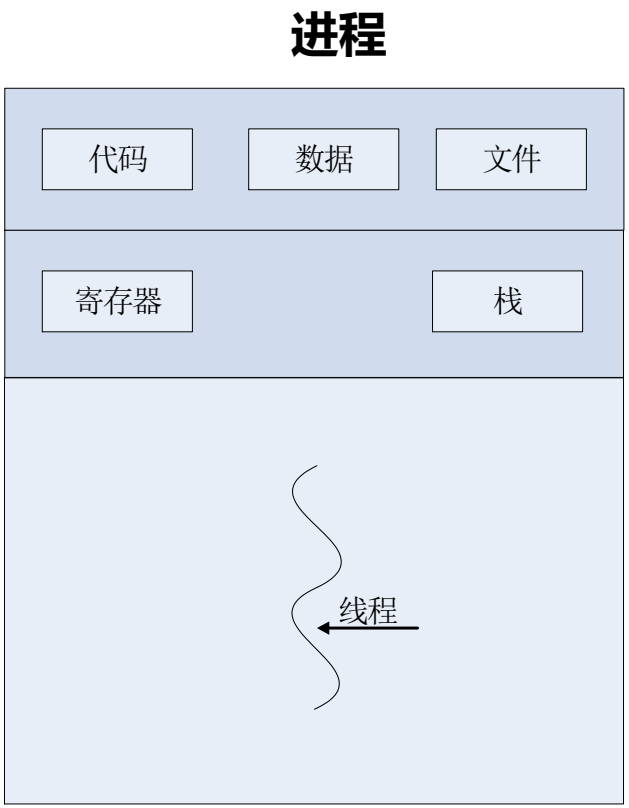


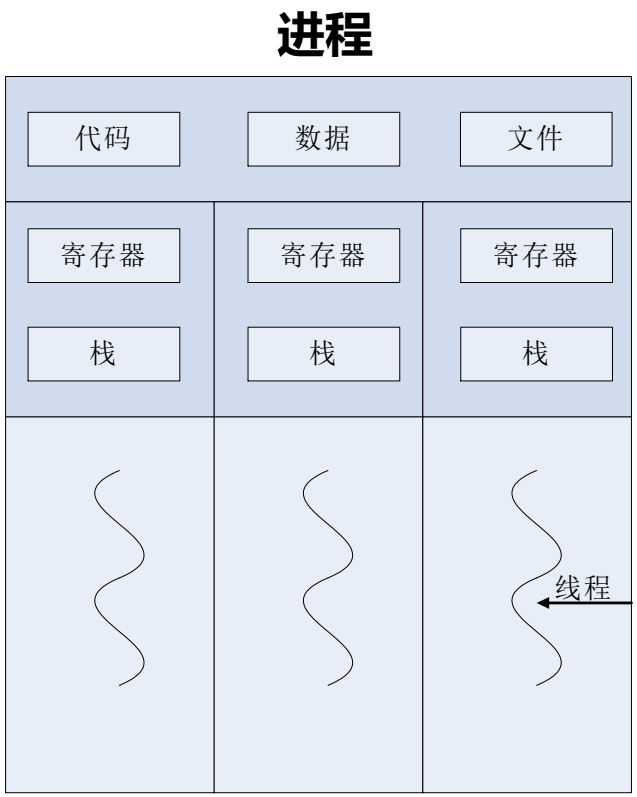
核1

核2

多线程

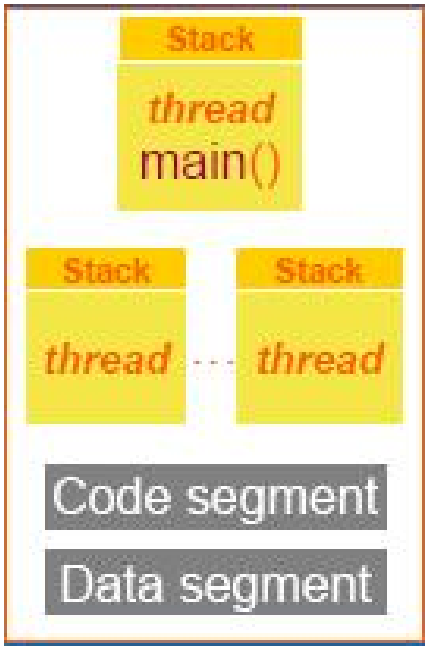
- 在单个程序中同时运行多个线程完成不同的工作，称为多线程





多线程

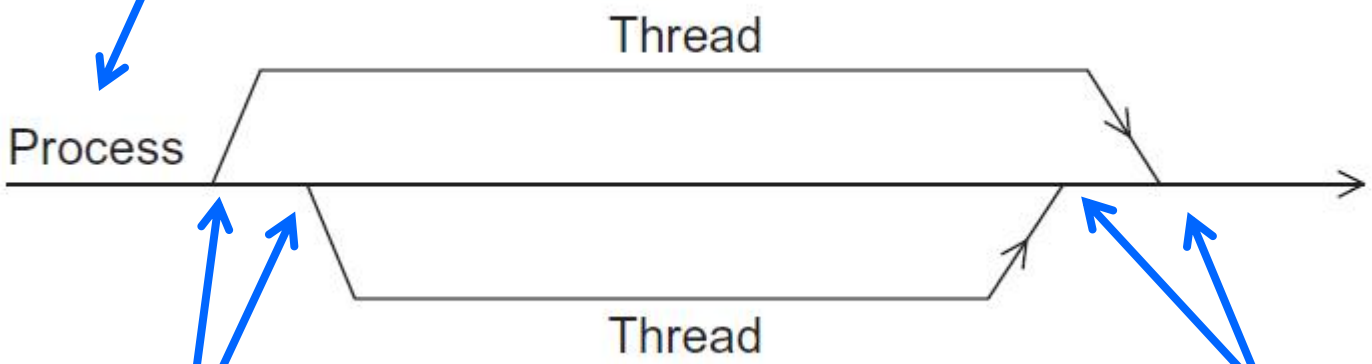
- 创建一个线程比创建一个进程的代价要小
- 线程的切换比进程间的切换代价小
- 充分利用多处理器
- 数据共享，使线程间的通信比进程间通信更高效



进程

多线程

the “master” thread（主线程）



starting a thread
Is called ***forking（派生）***

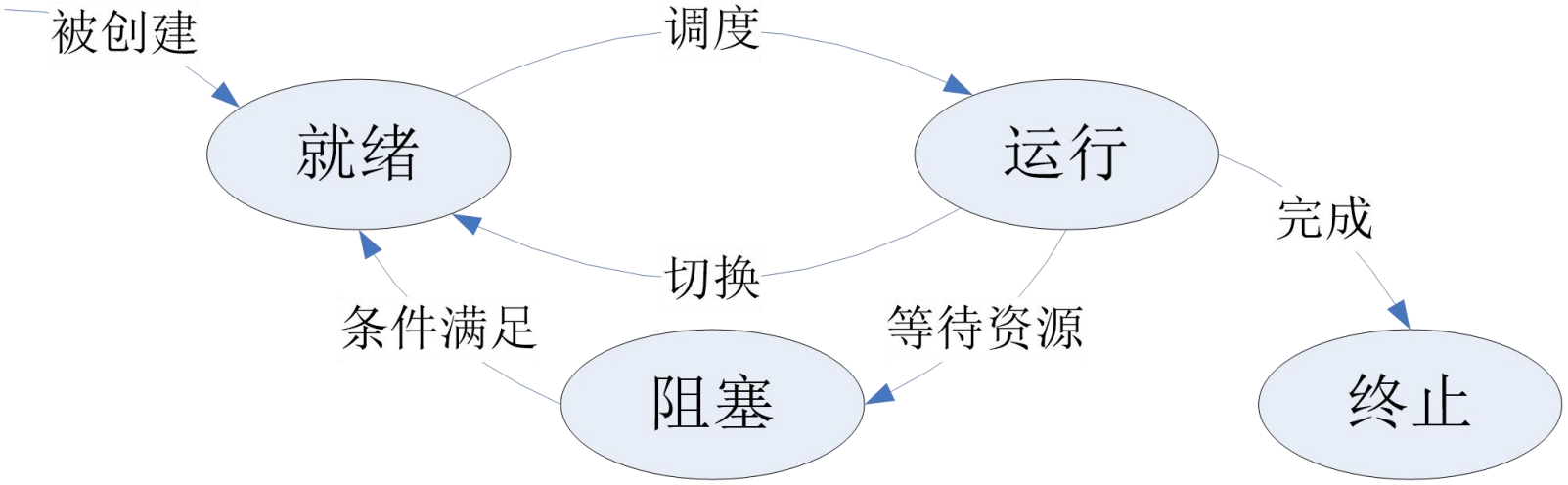
terminating a thread
Is called ***joining（合并）***

多线程

- 线程的生命周期
- 线程的标识：通常用一个整数来标识一个线程
- 线程的创建：
  - 自动创建，从main函数开始的主线程
  - 调用函数库接口创建新线程（pthread_create）
- 线程的终止
  - 执行完毕，或者调用了pthread_exit
  - 主线程退出导致整个进程终止

多线程

- **线程的状态**
  - **就绪（ready）：线程等待可用的处理器。**
  - **运行（running）：线程正在被执行。**
  - **阻塞（blocked）：线程正在等待某个事件的发生（比如I/O的完成，试图加锁一个被上锁的互斥量）。**
  - **终止（terminated）：线程从起始函数中返回或者调用pthread_exit。**



目录

- 进程与线程
- **基于Pthreads的多线程并行**
- 基于OpenMP的多线程并行

Pthreads库简介

- POSIX为支持多线程应用而提供的一组程序设计标准接口（API）
- 一个类Unix操作系统（如Linux、Mac OS等）上的标准库
- Pthread API提供了编写并行程序必须的函数，主要分为如下几类：
  - 线程管理（Thread management）
  - 互斥量（Mutexes）
  - 条件变量（Condition variables）
  - 同步（Synchronization）

“Hello world”（1）

- Pthreads程序示例：主函数启动了多个线程，每个线程打印一条消息，然后退出

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

/* Global variable: accessible to all threads */
int thread_count;

void * Hello(void* rank);                    /* Thread function*/

int main(int argc, char* argv[]){
   long thread;              /* Use long in case of a 64-bit system*/
   pthread_t* thread_handles;

   /* Get number of threads from command line*/
   thread_count = strtol(argv[1], NULL, 10);
   thread_handles = malloc(thread_count * sizeof(pthread_t));
```

“Hello world”（1）

- Pthreads程序示例：主函数启动了多个线程，每个线程打印一条消息，然后退出

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

ble to all threads */

/* Thr

int main(int argc, char* argv[]){
    long thread;            /* Use long in cas
    pthread_t* thre

    /* Get number of threads from command line*/
    thread_count = strtol(argv[1], NULL, 10);
    thread_handles = malloc(thread_count * sizeof(pthread_t));
```

pthread.h，这是Pthreads线程库的头文件，用来声明Pthreads的函数、常量和类型等

pthread_t数据结构用来存储线程的专有信息，由pthread.h声明。pthread_t对象是一个不透明对象。对象中存储的数据都是系统绑定的，用户级代码无法直接访问到里面的数据

全局变量，线程共享

从命令行读取需要生成的线程数量。strtol函数的功能是将字符串转化为long int（长整型），在stdlib.h中声明

为每个线程的pthread_t对象分配内存

“Hello world”（2）

- Pthreads程序示例：主函数启动了多个线程，每个线程打印一条消息，然后退出

```
    for (thread = 0; thread < thread_count ; thread++)
        pthread_create(&thread_handles[thread], NULL,
                Hello, (void*) thread);

    printf("Hello from the main thread\n");

    for (thread = 0; thread < thread_count; thread++)
        pthread_join(thread_handles[thread], NULL);

    free(thread_handles);

    return 0;
}   /* main */
```

“Hello world”（2）

- Pthreads程序示例：主函数启动了多个线程，每个线程打印一条消息，然后退出

```
for (thread = 0; thread < thread_count ; thread++)
    pthread_create(&thread_handles[thread], NULL,
            Hello, (void*) thread);

printf("Hello from the ma
```

调用pthread_create函数来生成线程

```
int pthread_join(pthread_t *thread,
                 void ** ret_val_p);
```

第二个参数可以接收任意由pthread_t对象所关联的那个线程产生的返回值

每个线程调用一次pthread_join函数。调用一次pthread_join将等待pthread_t对象所关联的那个线程结束

```
d_create(pthread_t *thread,
         pthread_attr_t *attr,
         void *(*start_fun)(void*),
         void * arg);
```

thread 操作系统返回的线程ID
attr 程序指定的线程属性 (或NULL)
start_fun 线程的开始函数
arg 线程开始函数的参数

“Hello world”（3）

- Pthreads程序示例：主函数启动了多个线程，每个线程打印一条消息，然后退出

```
void* Hello(void* rank){
    /* Use long in case of 64-bit system */
    long my_rank = (long) rank;

    printf("Hello from thread %ld of %d\n", my_rank,
                                            thread_count);

    return NULL;
} /* Hello Function */
```

“Hello world”（3）

- **Pthreads程序示例：主函数启动了多个线程，每个线程打印一条消息，然后退出**

```
void* Hello(void* rank){
    /* Use long in case of 64
    long my_rank = (long)

    printf("Hello from thr


    return NULL;
} /* Hello Function */
```

由pthread_create生成并运行的函数

void* **thread_function** (void* args_p)

void*　可以转换成C语言中任意指针类型
args_p　可以指向一个列表，该列表包含一个或者多个该函数需要的数值
thread_function返回的值 可以是一个包含一个或者多个值的列表

“Hello world”（4）

- **编译和运行Pthreads程序**

编译:

```
gcc pth_hello.c -o pth_hello -lpthread
```

运行:

```
./pth_hello   <number of threads>
```

```
./pth_hello 1↙
Hello from the main thread
Hello from thread 0 of 1
```

```
./pth_hello 4↙
Hello from the main thread
Hello from thread 0 of 4
Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 3 of 4
```

线程创建

向调用者传递子线程的线程号

```
int pthread_create (
        pthread_t    *thread_handle,
        pthread_attr_t      *attribute,
        void    * (*thread_function)(void *),
        void    *arg );
```

控制线程的各种属性

指定子线程需要执行的函数

子线程所运行的函数的参数值

```
typedef struct
{
    int                 detachstate;   //线程的分离状态
    int                 schedpolicy;  //线程调度策略
    structsched_param           schedparam;  //线程的调度参数
    int                 inheritsched;  //线程的继承性
    int                 scope;       //线程的作用域
    size_t              guardsize;   //线程栈末尾的警戒缓冲区大小
    int                 stackaddr_set;   //线程堆栈的地址集
    void*               stackaddr;   //线程栈的位置
    size_t              stacksize;    //线程栈的大小
} pthread_attr_t;
```

线程终止

- **阻塞并等待thread参数指定的线程的终止。通常由“master thread”调用**

```
int pthread_join ( pthread_t thread, void **ptr );
```

- **使线程正常终止，value_ptr是数据指针，可以储存线程的返回值，可以被pthread_join()函数的第二个参数接收，通常由“worker thread”调用**

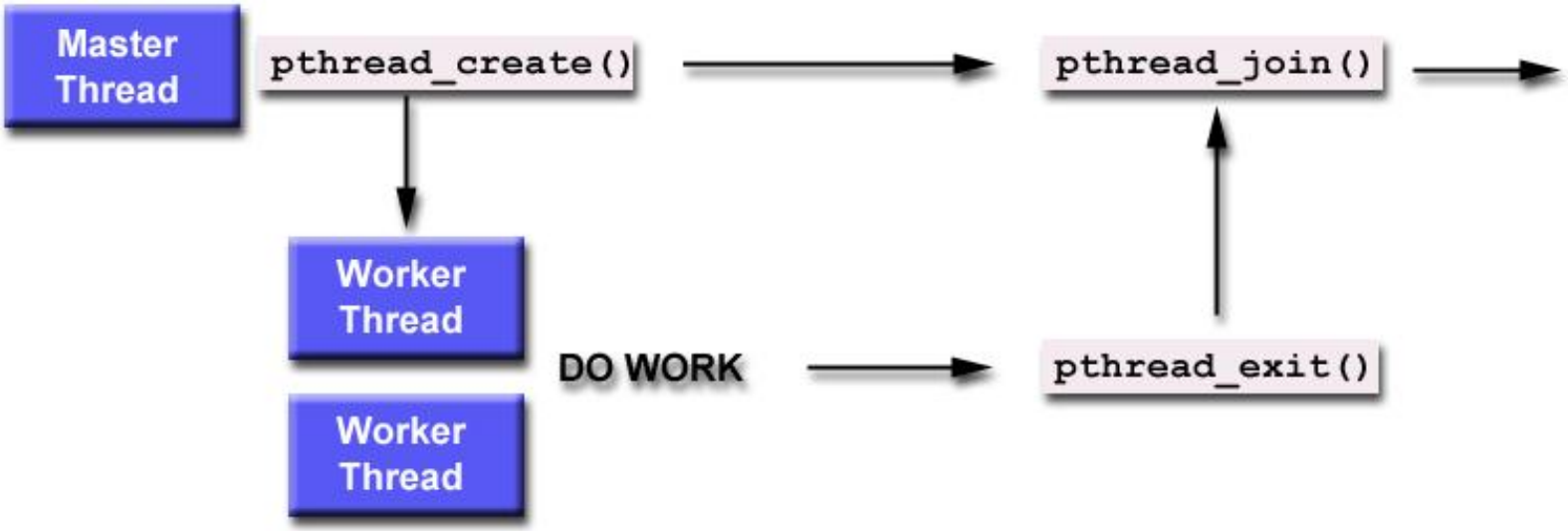
```
void pthread_exit ( void *value_ptr );
```

- **被同进程中的其它线程（包括主线程）强制终止**

```
int pthread_cancel ( pthread_t thread );
```

线程终止

- pthread_join实现线程间同步的一种方式，子线程合入主线程，主线程阻塞等待子线程结束，然后回收子线程资源



- 主线程与子线程分离，子线程结束资源自动回收

```
int pthread_detach ( pthread_t thread );
```

矩阵-向量乘法

- 如果A=(a_{ij})是一个m*n的矩阵，x是一个n维列向量，矩阵-向量的乘积Ax=y是个m维的列向量

$$
\begin{bmatrix}
a_{00} & a_{01} & \cdots & a_{0,n-1} \\
a_{10} & a_{11} & \cdots & a_{1,n-1} \\
\vdots & \vdots & & \vdots \\
a_{i0} & a_{i1} & \cdots & a_{i,n-1} \\
\vdots & \vdots & & \vdots \\
a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1}
\end{bmatrix}
\begin{bmatrix}
x_0 \\ x_1 \\ \vdots \\ x_{n-1}
\end{bmatrix}
=
\begin{bmatrix}
y_0 \\ y_1 \\ \vdots \\ y_i = a_{i0}x_0 + a_{i1}x_1 + \cdots a_{i,n-1}x_{n-1} \\ \vdots \\ y_{m-1}
\end{bmatrix}
$$

串行伪代码

```
/*For each row of A*/
for(i = 0; i < m; i++) {
    y[i] = 0.0;
    /*For each element of the row and each element of x*/
    for(j = 0; j < n; j++)
        y[i] += A[i][j] * x[j];
}
```

$$y_i = \sum_{j=0}^{n-1} a_{ij} x_j$$

并行化代码

- 通过把工作分配给各个线程将程序并行化
- 一种分配方法是将线程外层的循环分块，每个线程计算y的一部分
- 例如，假设m=n=6，线程数thread_count（或t）为3，则计算可以按下列情况分配

Thread	Components of y
0	y[0], y[1]
1	y[2], y[3]
2	y[4], y[5]

并行化代码

- 为了计算y[0]，线程0将执行代码

```
y[0] = 0.0;
for(j = 0; j < n; j++) {
        y[0] += A[0][j] * x[j];
}
```

- 线程0需要访问矩阵A的第0行以及向量x中的每一个元素
- 一般地，被分配给y[i]的线程将执行代码

```
y[i] = 0.0;
for(j = 0; j < n; j++) {
        y[i] += A[i][j] * x[j];
}
```

使用3个线程

每个线程除了访问各自分配到的矩阵A的第i行以及y分量外，还要访问X中的每个元素。这意味着最低限度下，要共享向量x

如果矩阵A和y是全局变量，主函数就可以简单地通过读取标准输入stdin来初始化矩阵A，乘积向量y也可以很容易被主线程打印输出

Thread	Components of y
0	y[0], y[1]
1	y[2], y[3]
2	y[4], y[5]

thread 0

```
y[0] = 0.0;
for(j = 0; j < n; j++) {
        y[0] += A[0][j] * x[j];
}
```

general case

```
y[i] = 0.0;
for(j = 0; j < n; j++) {
        y[i] += A[i][j] * x[j];
}
```

并行化代码

- 编写每一个线程的代码，确定每个线程计算哪一部分的y
- 假设，m与n都能被t整除，在例子中，m=6，t=3，每个线程能分配到m/t=3行的运算数据，而且，线程0处理第一部分的m/t行，线程1处理第二部分的m/t 行…
- 所以线程q处理的矩阵行是

第一行： $q \times \frac{m}{t}$

最后一行： $(q+1) \times \frac{m}{t} - 1$

$$
\begin{bmatrix}
a_{00} & a_{01} & \cdots & a_{0,n-1} \\
a_{10} & a_{11} & \cdots & a_{1,n-1} \\
\vdots & \vdots & & \vdots \\
a_{i0} & a_{i1} & \cdots & a_{i,n-1} \\
\vdots & \vdots & & \vdots \\
a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1}
\end{bmatrix}
\begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{n-1} \end{bmatrix}
=
\begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_i = a_{i0}x_0 + a_{i1}x_1 + \cdots a_{i,n-1}x_{n-1} \\ \vdots \\ y_{m-1} \end{bmatrix}
$$

Pthreads多线程实现

```
void* Path_mat_vec(void* rank) {
    y[i] = 0.0;
    long my_rank = (long) rank;
    int i, j;
    int local_m = m/thread_count;
    int my_first_row = my_rank * local_m;
    int my_last_row = (my_rank + 1) * local_m – 1;

    for(i = my_first_row; j <= my_last_row; i++) {
        y[i] = 0.0;
        for(j = 0; j < n; j++)
            y[i] += A[i][j] * x[j];
    }
    return NULL;
}/* Pth_mat_vect function*/
```

练习

- 编写完整的Pthreads矩阵-向量程序

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

/*Global variables*/
int        thread_count;
Int        m, n;
double*  A;
double*  x;
double*  y;

/*Serial functions*/
void Usage(char* prog_name);
void Read_matrix(char* prompt, double A[], int m, int n);
void Read_vector(char* prompt, double x[], int n);
void Print_matrix(char* title, double A[], int m, int n);
void Print_vector(char* title, double y[], double m);

/*Parallel function*/
void* Pth_mat_vect(void* rank);
```

提示

共享变量被改写?

- 共享内存区域是较理想的存储访问方式，所以矩阵-向量乘法的代码很容易编写
- 在主函数创建线程后，除了y以外，没有任何共享变量被改写
- 即使是y，也是每个线程各自改变属于自己运算的那一部分，没有两个或两个以上线程共同处理同一部分y的情况
- 如果多个线程需要更新同一内存单元的数据会怎样?

Thread	Components of y
0	y[0], y[1]
1	y[2], y[3]
2	y[4], y[5]

估计π值

$$\pi = 4\left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots + (-1)^n \frac{1}{2n+1} + \cdots\right)$$

```
double factor = 1.0;
double sum = 0.0;

for(i = 0; j < n; i++, factor = -factor) {
      sum += factor/(2 * I + 1);
}
pi = 4.0 * sum;
```

估计π值的并行化

- 用并行化矩阵-向量乘法的方法并行化该程序：将for循环分块后交给各个线程处理，并将sum设为全局变量
- 为了简化计算，假设线程数thread_count，简称t能够整除项目总数n。一般地，对于线程q，循环变量的范围是

$$\bar{n} = n/t$$

$$q\bar{n}, q\bar{n}+1, q\bar{n}+2, \cdots, (q+1)\bar{n}-1$$

使用线程函数来计算π

```
void* Thread_sum(void* rank) {
    long my_rank = (long) rank;
    double factor;
    long long i;
    long long my_n = n/thread_count;
    long long my_first_i = my_n * my_rank;
    long long my_last_i = my_first_i + my_n;

     if (my_first_i % 2 == 0) /*my_first_i is even*/

        factor = 1.0;
     else                        /*my_first_i is odd*/
        factor = -1.0;

    for(i = my_first_i; j < my_last_i; i++, factor = -factor) {
        sum += factor/(2 * i +1);
    }
    return NULL;
}/*Thread_sum function*/
```

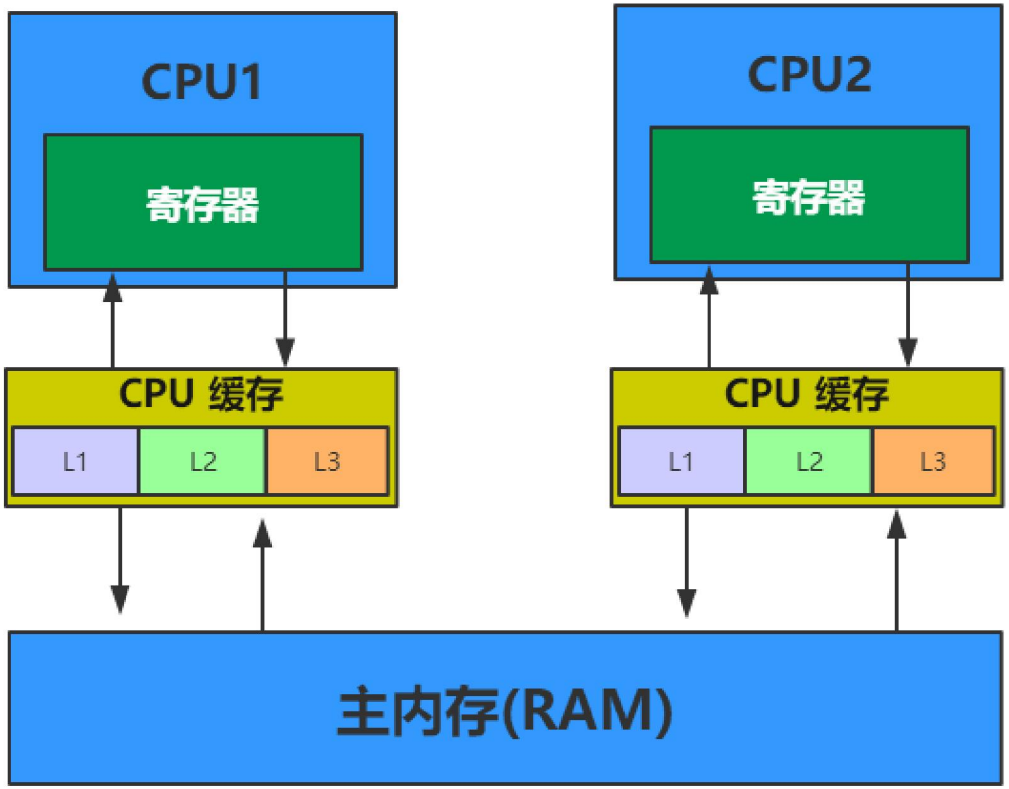
使用双线程测试

- 随着n的增加，单线程的估算结果也越来越准确，n每增大十倍就能获得更加精确的结果
- 双线程的计算结果反而变糟。事实上，多次运行双线程程序，尽管n未变，但每次的结果都不同
- 结论：当多个线程尝试更新同一个共享变量时，会出现问题

	n			
	10^5	10^6	10^7	10^8
π	3.14159	3.141593	3.1415927	3.14159265
1 Thread	3.14158	3.141592	3.1415926	3.14159264
2 Threads	3.14158	3.141480	3.1413692	3.14164686

可能的竞争条件

- 用一条C语言语句将存储单元y的内容加到存储单元x中去：x=x+y
- 机器执行：从主存中加载到CPU的寄存器，才能进行加法运算。当运算完成后，必须将结果再从寄存器重新存储到主存中



可能的竞争条件

- 假设有2个线程，每个线程对并存储在自己私有变量y中的值进行计算。还假设将这些私有变量加到共享变量x中，主线程将x的初始值置为0

- 每个线程执行以下代码

```
y= Compute(my_rank);
x=x+y;
```

- 假设线程0计算出的结果为y=1，线程1计算出的结果为y=2，则“正确”结果就应该是x=3

可能的竞争条件

y= Compute(my_rank);
x=x+y;

- 但是。。。

时间	线程0	线程1
t_1	被主线程启动	
t_2	调用Compute()	被主线程启动
t_3	赋值y=1	调用Compute()
t_4	将x=0和y=1放入寄存器	赋值y=2
t_5	将0和1相加	将x=0和y=2放入寄存器
t_6	把1存储在存储单元x	将0和2相加
t_7		把2存储在存储单元x

竞争条件

- 当多个线程尝试更新一个共享资源（共享变量）时，结果可能是无法预测的

- 当多个线程都要访问共享变量或共享文件这样的共享资源时，如果至少其中一个访问是更新操作，那么这些访问就可能会导致某种错误，我们称之为竞争条件(race condition)

临界区

- **临界区**：一个更新共享资源的代码段，一次只允许一个线程执行该代码段
- **目的**：当多个线程访问共享数据时，为保正数据的完整性，将共享数据保护起来
- **访问临界区的原则**：
  - 一次最多只能一个线程停留在临界区内
  - 不能让一个线程无限地停留在临界区内

临界区：忙等待

```
y = Compute(my_rank);
x = x + y;
```

临界区

```
flag = 0;  // 主线程初始化
……
y = Compute(my_rank);
while (flag != my_rank);
x = x + y;
flag++;
```

执行临界区代码的顺序：线程0，线程1，线程2...

使用忙等待求全局和（计算π）

```
void* Thread_sum(void* rank) {
    long my_rank = (long) rank;
    double factor;
    long long i;
    long long my_n = n/thread_count;
    long long my_first_i = my_n * my_rank;
    long long my_last_i = my_first_i + my_n;

     if (my_first_i % 2 == 0) /*my_first_i is even*/
        factor = 1.0;
     else                     /*my_first_i is odd*/
        factor = -1.0;

    for(i = my_first_i; j < my_last_i; i++, factor = -factor) {
        while (flag != my_rank);
        sum += factor/(2 * i +1);
        flag = (flag+1) % thread_count;
    }
    return NULL;
}/*Thread_sum function*/
```

问题?

使用忙等待求全局和（计算π）

```
void* Thread_sum(void* rank) {
    long my_rank = (long) rank;
    double factor;
    long long i;
    long long my_n = n/thread_count;
    long long my_first_i = my_n * my_rank;
    long long my_last_i = my_first_i + my_n;
    double my_sum = 0.0;

     if (my_first_i % 2 == 0) /*my_first_i is even*/
         factor = 1.0;
     else                     /*my_first_i is odd*/
         factor = -1.0;

    for(i = my_first_i; j < my_last_i; i++, factor = -factor) {
         my_sum += factor/(2 * i +1);
    }
     while (flag != my_rank);
     sum += my_sum;
     flag = (flag+1) % thread_count;
    return NULL;
}/*Thread_sum function*/
```

循环后用忙等待求全局和

临界区：互斥量

- 互斥量是互斥锁的简称，特殊类型的变量，通过某些特殊类型的函数，互斥量可以用来限制每次只有一个线程能进入临界区

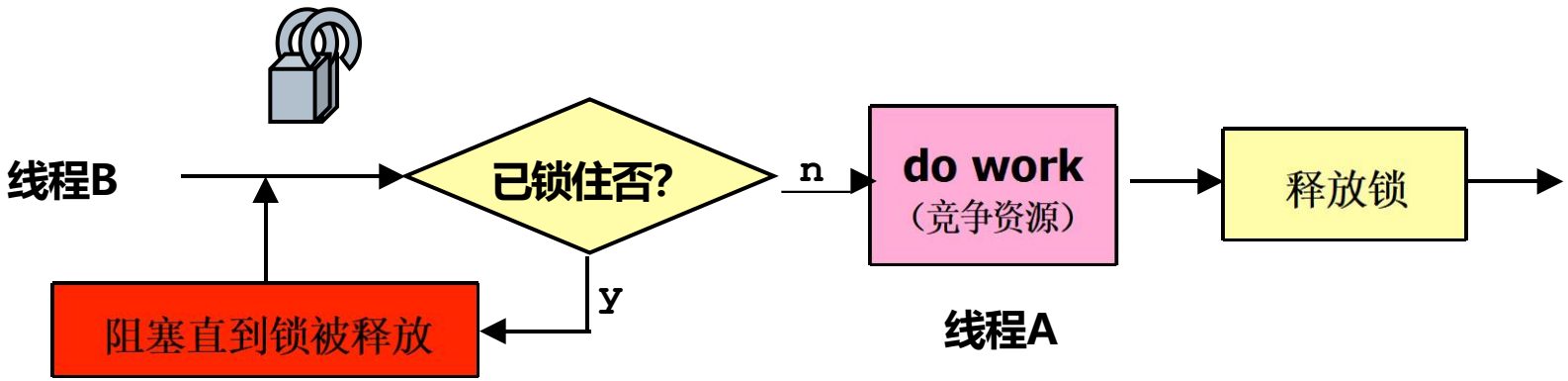
- 互斥量可以保证了一个线程独享临界区，其他线程在有线程以及进入该临界区的情况下，不能同时进入

临界区：互斥量

- 互斥量有两种状态：

  lock　互斥量一旦被某个线程锁住，其它申请该锁的线程将被阻塞，直至锁被解开

  unlock　未锁住的互斥锁允许线程通过，然后自动变为lock状态。已锁住的互斥锁必须由得到此锁的线程释放



临界区：互斥量

- Pthread标准为互斥量提供了一个特殊类型 pthread_mutex_t

- 创建一个互斥量，其状态为unlock

```
pthread_mutex_t   mtx;
pthread_mutex_t mymutex = PTHREAD_MUTEX_INITIALIZER;
或
mtx = pthread_mutex_init(&mtx, NULL);
```

临界区：互斥量

- 加锁（阻塞和非阻塞）

如果状态为unlock，则得到此锁并改变状态为lock，然后返回； 否则阻塞线程

```
pthread_mutex_lock(pthread_mutex_t *mtx);
pthread_mutex_trylock(pthread_mutex_t *mtx);
```

- 解锁

改变锁状态为unlock，然后返回

```
pthread_mutex_unlock(pthread_mutex_t *mtx);
```

- 销毁互斥量

程序使用完互斥量后，进行销毁

```
pthread_mutex_destroy(pthread_mutex_t *mtx);
```

使用互斥量求全局和（计算π）

```
void* Thread_sum(void* rank) {
    long my_rank = (long) rank;
    double factor;
    long long i;
    long long my_n = n/thread_count;
    long long my_first_i = my_n * my_rank;
    long long my_last_i = my_first_i + my_n;
    double my_sum = 0.0;

     if (my_first_i % 2 == 0) /*my_first_i is even*/
         factor = 1.0;
     else                     /*my_first_i is odd*/
         factor = -1.0;

    for(i = my_first_i; j < my_last_i; i++, factor = -factor) {
         my_sum += factor/(2 * i +1);
    }
    pthread_mutex_lock(&mutex);
    sum += my_sum;
    pthread_mutex_unlock(&mutex);
    return NULL;
}/*Thread_sum function*/
```

注意：在使用互斥量的多线程程序中，多个线程进入临界区的顺序是随机的

练习

- **利用互斥量编写求π值的Pthread程序**

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <math.h>

/*Global variables*/
long        thread_count;
double    sum;
pthread_mutex_t mutex;

/*Serial functions*/
double Serial_pi(long long n);

/*Parallel function*/
void* Thread_sum(void* rank);
```

提示

main函数中对mutex进行初始化
pthread_mutex_init(&mutex, NULL);

main函数结束之前对mutex进行销毁
pthread_mutex_destroy(&mutex);

比较忙等待和互斥量

Programs Using $n = 10^8$ Terms on a System with Two Four-Core Processors

Threads	Busy-Wait	Mutex
1	2.90	2.90
2	1.45	1.45
4	0.73	0.73
8	0.38	0.38
16	0.50	0.38
32	0.80	0.40
64	3.56	0.38

$$\frac{T_{\text{serial}}}{T_{\text{parallel}}} \approx \texttt{thread_count}$$

当线程数多于核数时候，忙等待效率降低

- ✓ **忙等待：强调访问临界区的顺序**
- ✓ **互斥量：访问临界区的顺序随机的**

互斥量的问题

- 忙等待：能事先确定线程执行临界区代码的顺序
- 互斥量：哪个线程先进入临界区以及此后的顺序由系统随机选取
- 有一些应用，需要控制线程进入临界区的顺序

互斥量的问题

```
/*n and product_matrix are shared and initialized by the main
thread*/
/* product_matrix is initialized to be the identity matrix */
void* Thread_work(void* rank) {
    long my_rank = (long) rank;
    matrix_t my_mat = Allocate_matrix(n);
    Generate_matrix(my_mat);
    pthread_mutex_lock(&mutex);
    Multiply_matrix(product_mat, my_mat);
    pthread_mutex_unlock(&mutex);
    Free_matrix(&my_mat);
    return NULL;
}
```

矩阵乘法没有交换律
所以不能使用互斥量

信号量

- 信号量（Semaphore）可以用于临界区，保护共享资源
- 信号量的特性如下：
  - 信号量是一种特殊类型的unsigned int无符号整型变量，可以赋值为0、1、2、…
  - 要访问共享资源的线程必须获取一个信号量，则信号量减1
  - 当信号量为0时，试图访问共享资源的线程将处于等待状态
  - 离开共享资源的线程释放信号量，信号量加1

信号量

12个停车位，空3个位置，来了5辆车

信号量

```
include <semaphore.h>
```

参数1：信号量指针
参数2：信号量类型（一般设置为0）
参数3：信号量初始值

- **信号量初始化**

```
int sem_init(sem_t* semaphore_p, int shared,
                unsigned initial_val);
```

- **销毁信号量**

```
int sem_destroy(sem_t* semaphore_p);
```

释放一个信号量，信号量的值加1

- **发信号**

```
int sem_post(sem_t* semaphore_p);
```

申请一个信号量，当前无可用信号量则等待，有可用信号量时占用一个信号量，对信号量的值减1

- **等待信号**

```
int sem_wait(sem_t* semaphore_p);
```

例：线程发送消息

- 假设t个线程，线程0向线程1发消息，线程1向线程2发消息，。。。，线程t-2向线程t-1发消息，线程t-1向线程0发消息
- 一个线程“接收”一条消息后，打印消息并终止
- 为实现消息传递，分配一个char*类型的共享数组，每个线程初始化消息后，共享数组的指针指向要发送的消息
- 主线程把共享数组messages初始化为NULL
- 主线程把信号量初始化为0

例：线程发送消息

```
main函数中对信号量进行初始化
semaphores = (sem_t*)malloc(thread_count*sizeof(sem_t));
for(thread=0; thread < thread_count; ++thread) {
    sem_init(&semaphores[thread],0,0);
}
```

```
void*  Send_msg(void*  rank) {
    long my_rank = (long) rank;
    long dest = (my_rank + 1) % thread_count;
    char*  my_msg = malloc( MSG_MAX * sizeof(char) );
    sprintf(my_msg, "Hello to %ld from %ld", dest, my_rank);
    messages[dest] = my_msg;
    sem_post(&semaphores[dest]);

    sem_wait(&semaphores[my_rank]);
    printf("Thread %ld > %s\n", my_rank, messages[my_rank]);

    return NULL;
} /*  Send_msg*  /
```

信号量

- 信号量与互斥量最大的区别在于信号量是没有个体拥有权的，主线程将所有的信号量初始化为0，即“加锁”，其他线程都能对任何信号量调用sem_post和sem_wait函数

路障

- 同步点（Barrier，屏障/路障）：线程到达路障进入阻塞状态，等待所有线程都到达，才能继续执行
- 屏障的应用：
  - 计时
  - 调试程序
  - 。。。

利用路障计算“最慢”线程的时间

```
/*  Shared * /
double elapsed_time;
. . .
/*  Private * /
double my_start, my_finish, my_elapsed;
. . .
Synchronize threads;
Store current time in my_start;
/*  Execute timed code * /
. . .
Store current time in my_finish;
my_elapsed = my_finish - my_start;

elapsed_time = Maximum of my_elapsed values;
```

利用路障调试程序

```
point in program we want to reach;
barrier;
if (my_rank == 0) {
    printf("All threads reached this point\n");
    fflush(stdout);
}
```

路障

```
include <pthread.h>
```

参数1：barrier对象地址
参数2：barrier的属性对象
参数3：需要等待的线程数量

- **路障初始化**

```
int pthread_barrier_init(pthread_barrier_t *restrict barrier,
        pthread_barrierattr_t *restrict attr, unsigned count);
```

- **等待路障**

要等待的barrier

```
int pthread_barrier_wait(pthread_barrier_t *barrier);
```

- **销毁路障**

要销毁的barrier

```
int pthread_barrier_destroy(pthread_barrier_t *barrier);
```

路障示例

```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <unistd.h>
pthread_barrier_t barrier;

void* initor(void* args)
{
    printf("----thread init work(%d)----\n", time(NULL));
    // 模拟初始化工作。
    sleep(10);
    // 到达路障
    pthread_barrier_wait(&barrier);
    printf("----thread start work(%d) ----\n", time(NULL));
    sleep(10);
    printf("----thread stop work(%d) ----\n", time(NULL));
    return NULL;
}
```

路障示例

```
****main thread barrier init done****
****main waiting(1696299436)****
----thread init work(1696299436)----
----thread start work(1696299446) ----
****main start to work(1696299446)****
----thread stop work(1696299456) ----
****thread complete(1696299476)****
```

```
int main(int argc, char* argv[])
{
    // 初始化路障，该路障等待两个线程到达时放行
    pthread_barrier_init(&barrier, NULL, 2);

    printf("****main thread barrier init done****\n");
    pthread_t pid;
    pthread_create(&pid, NULL, &initor, NULL);
    printf("****main waiting(%d)****\n", time(NULL));

    // 主线程到达，被阻塞，当初始化子线程到达路障时才放行。
    pthread_barrier_wait(&barrier);

    pthread_barrier_destroy(&barrier);
    printf("****main start to work(%d)****\n", time(NULL));
    sleep(30);
    pthread_join(pid, NULL);
    printf("****thread complete(%d)****\n", time(NULL));
    return 0;
}
```

条件变量

- 条件变量（Conditional Varialble）使线程在某个特定条件或事件发生前处于挂起状态
- 条件变量数据类型

  pthread_cond_t

- 条件变量初始化

```
int pthread_cond_init(pthread_cond_t* cond_var_p,
                      const pthread_condattr_t* attr)
```

- 销毁条件变量

```
int pthread_cond_destroy (pthread_cond_t* cond_var_p)
```

条件变量

- **解锁一个阻塞的线程**

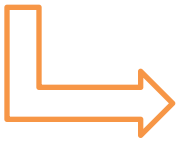
```
int pthread_cond_signal(pthread_cond_t*  cond_var_p);
```

- **解锁所有被阻塞的线程**

```
int pthread_cond_broadcast(pthread_cond_t*
cond_var_p);
```

- **通过互斥量阻塞线程**

```
int pthread_cond_wait( pthread_cond_t*  cond_var_p,
                       pthread_mutex_t*  mutex_p);
```



```
pthread_mutex_unlock(&mutex_p);
wait_on_signal(&cond_var_p);
pthread_mutex_lock(&mutex_p);
```

条件变量示例

```
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
#include <stdlib.h>

pthread_mutex_t Mutex1,Mutex2;
pthread_cond_t cond;

void *pthread_Task1(void *arg) {
  int cnt = 1;
  while(1) {
    pthread_mutex_lock(&Mutex1);
    printf("Task1...\r\n");
    if(cnt++ % 4 == 0)
      //pthread_cond_signal(&cond);
      pthread_cond_broadcast(&cond);
    pthread_mutex_unlock(&Mutex1);
    sleep(1);
  }
}
```

条件变量示例

```
void *pthread_Task2(void *arg) {
  while(1) {
    pthread_cond_wait(&cond,&Mutex2);
    printf("Task2...\r\n");
  }
}
int main(int argv,char **argc) {
  pthread_t pthread_id1,pthread_id2;

  pthread_mutex_init(&Mutex1,NULL);
  pthread_mutex_init(&Mutex2,NULL);
  pthread_cond_init(&cond,NULL);

  pthread_create(&pthread_id1,NULL,pthread_Task1,NULL);
  pthread_create(&pthread_id2,NULL,pthread_Task2,NULL);

  pthread_join(pthread_id1,NULL);
  pthread_join(pthread_id2,NULL);
  return 0;
}
```

```
Task1...
Task1...
Task1...
Task1...
Task2...
Task1...
Task1...
Task1...
Task1...
Task2...
Task1...
Task1...
Task1...
Task1...
Task1...
Task2...
Task1...
Task1...
Task1...
Task1...
Task2...
```

读写锁

- 读写锁 (Read Write Lock) 把对共享资源的访问者分为读者和写者，读者仅仅对共享资源进行读访问，写者仅仅对共享资源进行写操作
- 读写锁数据类型

  pthread_rwlock_t

- 初始化

```
int pthread_rwlock_init(pthread_rwlock_t*  rwlock_p,
                        const pthread_rwlockattr_t*  attr_p );
```

- 读加锁

```
int pthread_rwlock_rdlock(pthread_rwlock_t*  rwlock_p);
```

读写锁

- **写加锁**

```
int pthread_rwlock_wrlock(pthread_rwlock_t*  rwlock_p);
```

- **解锁**

```
int pthread_rwlock_unlock(pthread_rwlock_t*  rwlock_p);
```

- **销毁**

```
int pthread_rwlock_destroy(pthread_rwlock_t*  rwlock_p
);
```

读写锁示例

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <errno.h>

#define MAXDATA     1024
#define MAXREDER    100
#define MAXWRITER   100
struct {
   pthread_rwlock_t   rwlock;
   char datas[MAXDATA];
} shared = {
   PTHREAD_RWLOCK_INITIALIZER
};

void *reader(void *arg);
void *writer(void *arg);
```

读写锁示例

```
int main(int argc,char *argv[]) {
    int i,readercount,writercount;
    pthread_t tid_reader[MAXREDER],tid_writer[MAXWRITER];
    if(argc != 3) {
        printf("usage : <reader_writer> #<readercount> #<writercount>\n");
        return 0;
    }
    readercount = atoi(argv[1]);
    writercount = atoi(argv[2]);
    for(i = 0; i < writercount; ++i)
        pthread_create(&tid_writer[i], NULL, writer, NULL);
    sleep(10);
    for(i = 0; i < readercount; ++i)
        pthread_create(&tid_reader[i], NULL, reader, NULL);
    for(i = 0; i < writercount; ++i)
        pthread_join(tid_writer[i], NULL);
    for(i = 0; i < readercount; ++i)
        pthread_join(tid_reader[i], NULL);
    return 0;
}
```

读写锁示例

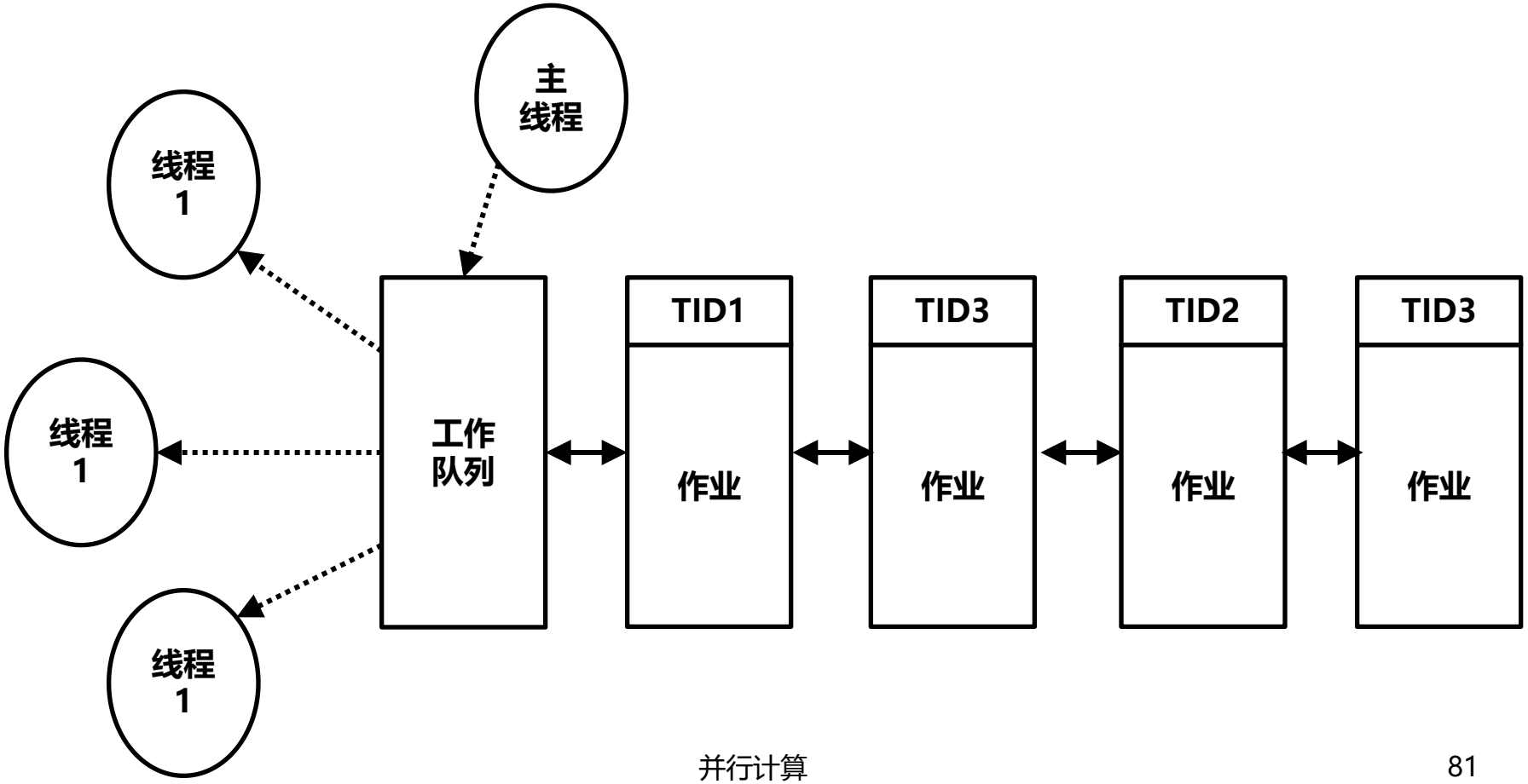
```
void *reader(void *arg) {
   pthread_rwlock_rdlock(&shared.rwlock);
   printf("Reader begins read message.\n");
   sleep(1);
   printf("Read message is: %s\n",shared.datas);
   pthread_rwlock_unlock(&shared.rwlock);
   return NULL;
}

void *writer(void *arg) {
   char datas[MAXDATA];
   pthread_rwlock_wrlock(&shared.rwlock);
   printf("Writers begings write message.\n");
   printf("Enter the write message: \n");
   gets(datas);
   strcat(shared.datas,datas);
   pthread_rwlock_unlock(&shared.rwlock);
   return NULL;
}
```

读写锁示例

```
./pth_rwlock_test 2 2↙
Writers begings write message.
Enter the write message:
hello↙
Writers begings write message.
Enter the write message:
world↙
Reader begins read message.
Reader begins read message.
Read message is: helloworld
Read message is: helloworld
```

读写锁应用场景

- 作业请求队列由单个读写锁保护，多个工作线程获取单个主线程分配给它们的作业



Pthreads小结

- 线程管理，创建、终止、分离、同步
- 临界区，互斥量，信号量
- 路障，条件变量，读写锁
- 扩展阅读：
  - POSIX Threads Programming, by Blaise Barney, Lawrence Livermore National Laboratory: https://computing.llnl.gov/tutorials/pthreads/

目录

- 进程与线程
- 基于Pthread的多线程并行
- **基于OpenMP的多线程并行**

OpenMP简介

https://www.openmp.org

- 通过在源代码（串行）中添加 OpenMP 指令和调用OpenMP库函数来实现在共享内存系统上的并行
- 为共享内存并行程序员提供了一种简单灵活的开发并行应用的接口模型，使程序既可以在台式机执行，也可以在超级计算机执行，具有良好的可移植性

Jointly defined by a group of major computer hardware and software vendors and major parallel computing user facilities, the OpenMP API is a portable, scalable model that gives shared-memory parallel programmers a simple and flexible interface for developing parallel applications on platforms ranging from embedded systems and accelerator devices to multicore systems and shared-memory systems. https://www.openmp.org

API：Application Programming Interface

OpenMP使用说明

- FORTRAN/C/C++ 自带程序库，无需另外安装
- 编译时不打开OpenMP编译选项，则编译器将忽略 OpenMP指令，从而生成串行可执行程序（串行等价性）
- 打开OpenMP编译选项，编译器将OpenMP指令进行处理，编译生成OpenMP并行执行程序
- 并行线程数可以在程序启动时利用环境变量等方法进行动态设置
- 编程方式：增量并行
- 支持与 MPI 混合编程

OpenMP发展历史

- 起源于ANSI X3H5（1994）草案，1997年，部分设备商和编译器开发商组成ARB（架构审查委员会），着手制定OpenMP标准化规范
- 目标：编程简单，增量化并行，移植性好，扩展性好，支持主流编译器
- 支持 Unix，Linux，Windows 等操作系统
- ARB成员：AMD，ARM，Intel，IBM，Cray，HP，NVIDIA，...

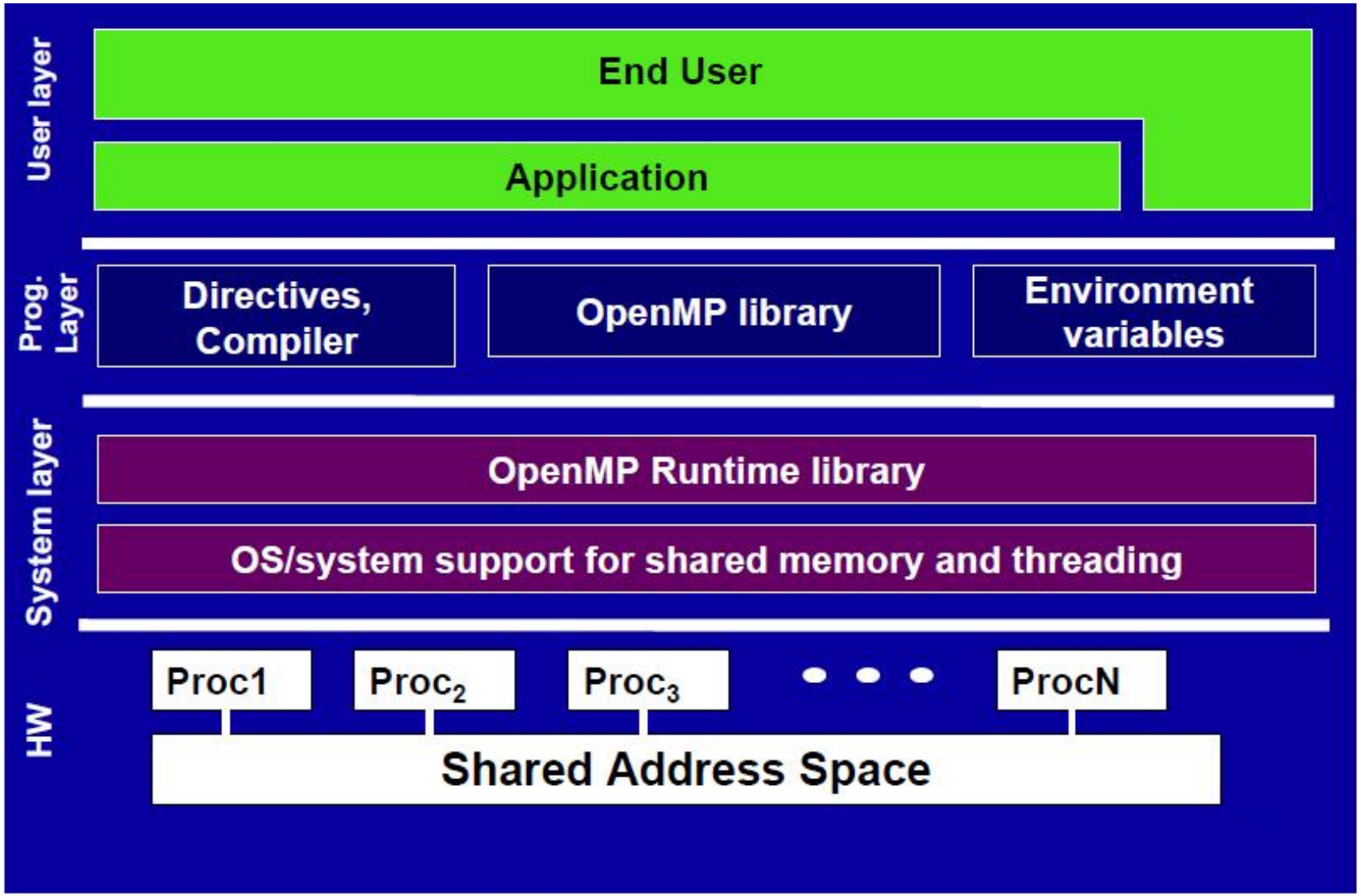
- FORTRAN version 1.0 (1997)，C/C++ version 1.0 (1998)
- FORTRAN version 2.0 (2000)，C/C++ version 2.0 (2002)
- OpenMP 3.0 – (May 2008)
- OpenMP 3.1 – (July 2011)
- OpenMP 4.0 – (July 2013)
- OpenMP 4.5 – (Nov 2015)
- OpenMP 5.0 – (Nov 2018)
- OpenMP 5.2 – (Nov 2021)

OpenMP

OpenMP
Application Programming
Interface

Version 5.2 November 2021

OpenMP组成


User layer
End User
Application
Prog. Layer
Directives, Compiler
OpenMP library
Environment variables
System layer
OpenMP Runtime library
OS/system support for shared memory and threading
HW
Proc1
Proc2
Proc3
ProcN
Shared Address Space

OpenMP并行方式

OpenMP是基于线程的并行编程模型

OpenMP采用Fork-Join并行执行方式

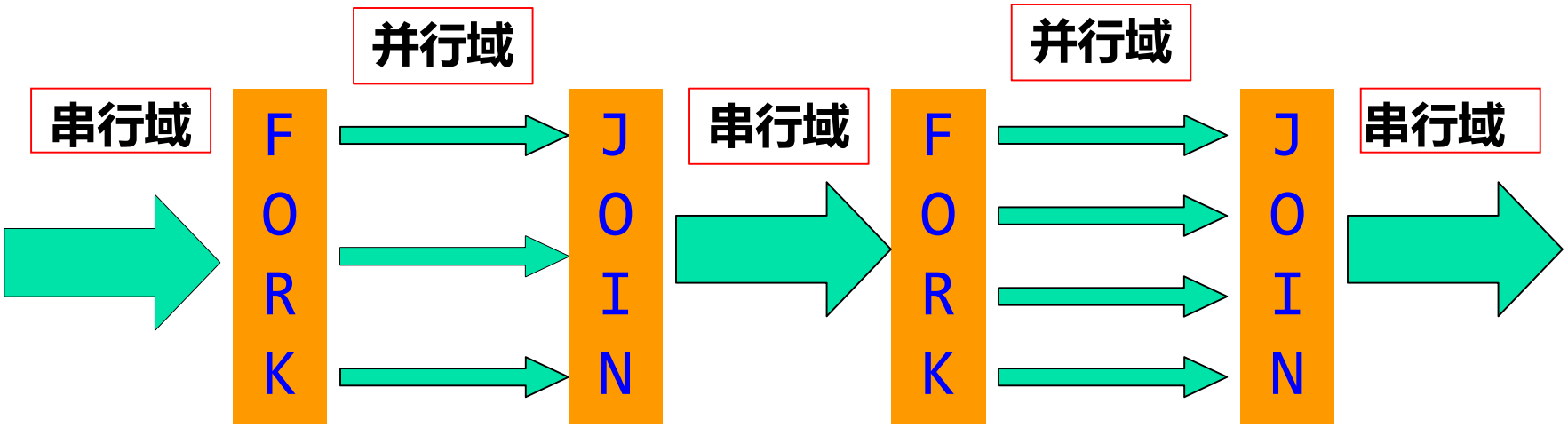
① OpenMP程序开始于一个单独的主线程（Master Thread），然后主线程一直串行执行（串行域）

② 直到遇见第一个并行域（Parallel Region），然后开始并行执行并行域

③ 并行域代码执行完后再回到主线程，执行串行域，直到遇到下一个并行域

④ 以此类推，直至程序运行结束。

Fork-Join



- **Fork**：主线程创建一个并行线程队列，然后并行域中的代码在不同的线程上并行执行
- **Join**：当并行域执行完之后，它们或被同步，或被中断，最后只有主线程继续执行

† 并行域可以嵌套

“Hello world”

- 头文件：omp.h
- OpenMP 指令标识符#pragma omp

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
void Hello(void); /*Thread function */

int main(int argc, char* argv[]) {
  /* Get number of threads from command line */
  int thread_count = strtol(argv[1], NULL, 10);

  #pragma omp parallel num_threads(thread_count)
  Hello();

  return 0;
} /* main */

void Hello(void) {
  int my_rank = omp_get_thread_num();
  int thread_count = omp_get_num_threads();
  printf("Hello from thread %d of %d\n", my_rank, thread_count);
} /* Hello */
```

- 编译

  gcc -fopenmp omp_hello.c -o omp_hello

“Hello world”

- **编译和运行OpenMP程序**

编译:

```
gcc -fopenmp omp_hello.c -o omp_hello
```

运行:

```
./omp_hello   <number of threads>
```

```
./omp_hello 4↙
Hello from thread 1 of 4
Hello from thread 0 of 4
Hello from thread 3 of 4
Hello from thread 2 of 4
Hello from thread 0 of 4
```

```
./omp_hello 4↙
Hello from thread 0 of 4
Hello from thread 1 of 4
Hello from thread 2 of 4
Hello from thread 3 of 4
Hello from thread 0 of 4
```

注意：线程正在竞争访问标准输出，因此不保证输出会按线程编号的顺序出现

几点说明

- OpenMP程序编写
  - 通常采用增量并行方法：逐步改造现有的串行程序，每次只对部分代码进行并行化，这样可以逐步改造，逐步调试
  - C/C++ 的 OpenMP 指令标识符为 #pragma omp
  - C/C++ 程序中，OpenMP指令区分大小写
  - OpenMP 指令后是一个结构块（用大括号括起来）
- 源程序编译

```
gcc -fopenmp omp_hello.c -o omp_hello
icc -openmp omp_hello.c -o omp_hello      // Intel C
```

OpenMP编程三要素

- 编译制导指令（Compiler Directive）
- 运行时库函数（Runtime Library Routines）
- 环境变量（Environment Variables）

要素1：编译制导指令

- OpenMP的大部分语句都是编译制导指令

Fortran	`!$OMP construct [clause [clause]…]`
C/C++	`#pragma omp construct [clause [clause]…]`

#pragma omp	construct	clause
制导指令前缀 所有的OpenMP制导指令语句都有前缀	制导指令 具体的指令，指导编译器并行化	子句 制导指令之后，负责添加一些补充设置

- 不支持OpenMP的编译器也能编译OpenMP程序，只是会忽略OpenMP语句

编译制导指令

OpenMP通过对串行程序添加编译制导指令实现并行化

编译制导指令分类

- **并行域指令**：创建并行域，即产生多个线程以并行方式执行任务，所有并行任务必须放在并行域中才能被并行执行
- **工作共享指令**：负责任务划分，并分发给各个线程，工作共享指令不能产生新线程，因此必须位于并行域中
- **同步指令**：负责并行线程之间的同步
- **数据环境**：负责并行域内的变量的属性（共享或私有），以及边界上（串行域与并行域）的数据传递

并行域：PARALLEL结构

■ 并行域的创建

```
#pragma omp parallel   // 创建并行域
```

- 产生多个线程，即生成一个并行域
- 并行域中的所有代码默认都将被所有线程并行执行
- 可以通过线程 id给不同线程手工分配不同的任务
- 也可以利用工作共享指令给每个线程分配任务
- 并行域可以嵌套
- 并行域结束后，将回到主线程

并行域：PARALLEL结构

Fortran	!$omp parallel [clause clause ...] *structured-block* !$omp end parallel

C/C++	#pragma omp parallel [clause clause ...]
	{
	structured-block
	}

- **结尾处有隐式同步，可用的子句包括：**

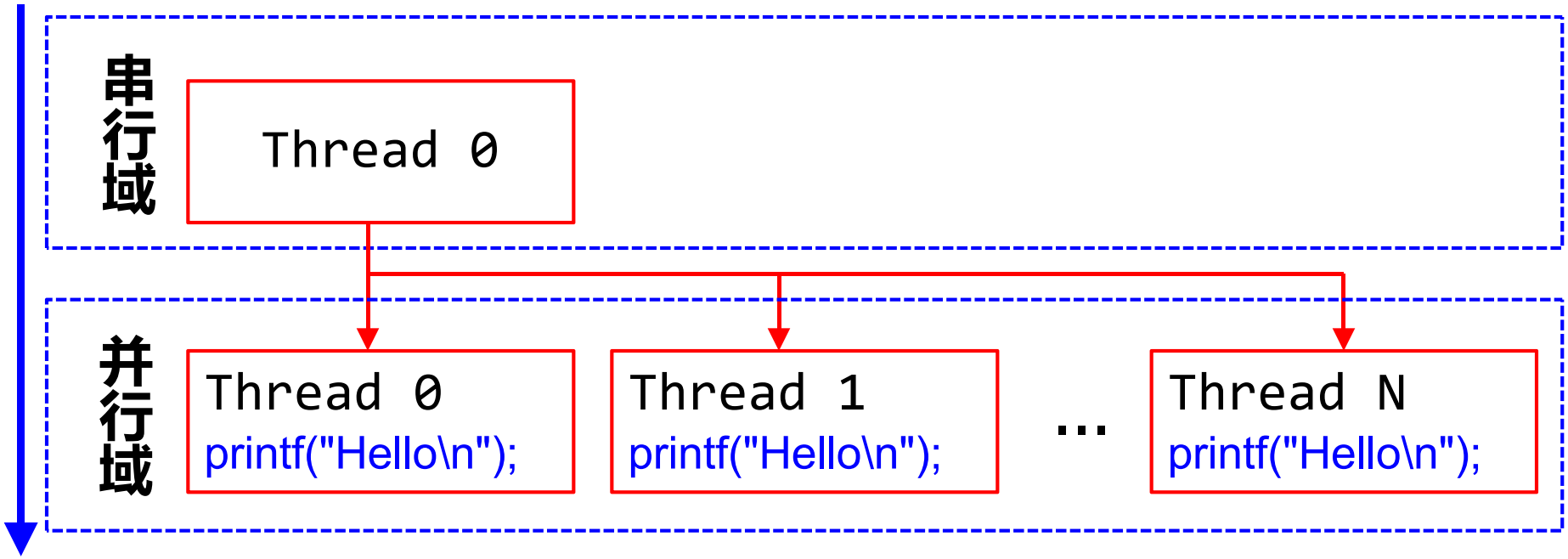
if (*scalar-logical-expression*)
num_threads(*scalar-integer-expression*)
default(shared | none)
private(*list*), **firstprivate**(*list*)
shared(*list*)
copyin(*list*)
reduction(*op* : *list*)

子句用来添加一些补充信息。若有多个，则用空格隔开。

若没有指定线程个数，则产生最大可能的线程个数。

并行域示例

```
#pragma omp parallel
{
    printf("Hello\n");
}
```



并行域示例

- 创建一个4-线程的并行域

每个线程都执行omp语句后面的代码块

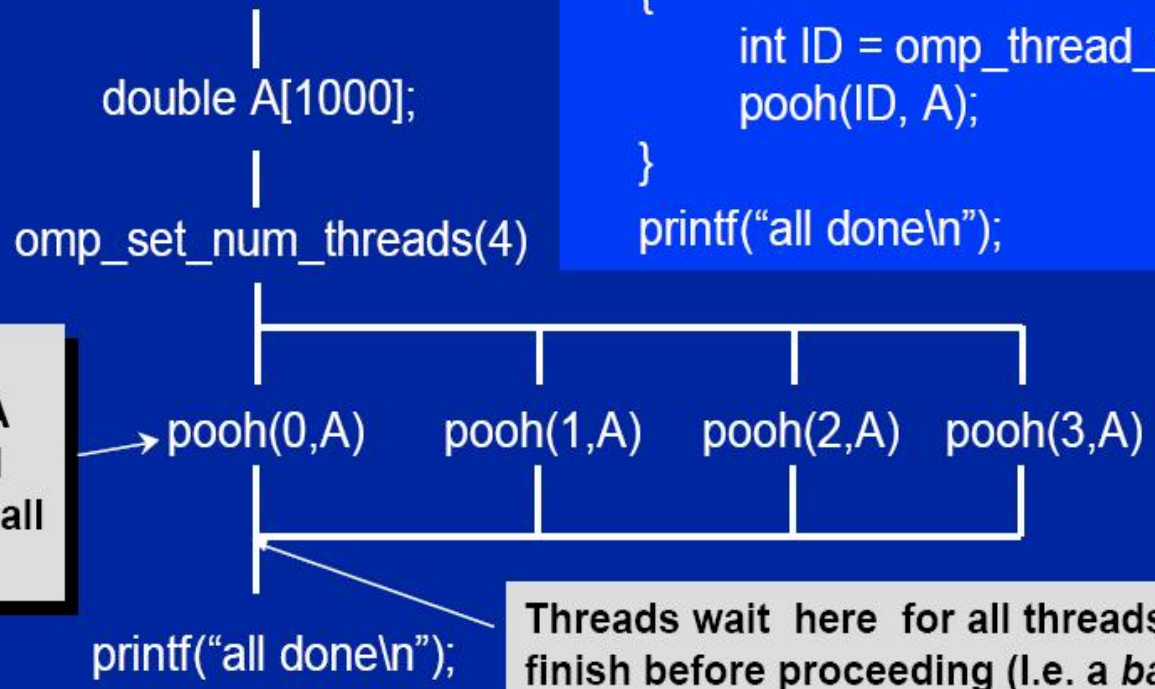
```
double A[1000];
omp_set_num_threads(4);
#pragma omp parallel
{
   int ID =omp_get_thread_num();
   pooh(ID,A);
}
printf("all done\n");
```

每个线程调用函数pooh(ID,A), ID = 0 to 3

并行域示例

- **Each thread executes the same code redundantly.**

```
double A[1000];
omp_set_num_threads(4);
#pragma omp parallel
{
        int ID = omp_thread_num();
        pooh(ID, A);
}
printf("all done\n");
```



A single copy of A is shared between all threads.

Threads wait here for all threads to finish before proceeding (I.e. a *barrier*)

并行域线程数设置

- **通过环境变量设置线程数**

```
export OMP_NUM_THREADS=4
```

- **在代码中设置线程数**

```
omp_set_num_threads(number_of_threads);
```

或

```
#pragma omp parallel num_threads(number_of_threads)
```

工作共享

- 工作共享指令 Work-Sharing Constructs

  - 负责任务的划分和分配，
  - 在每个工作分享结构入口处无需同步
  - 每个工作分享结构结束处会隐含路障同步

  - for指令：自动划分和分配循环任务
  - sections指令：手动划分任务
  - single指令：指定并行域中的串行任务
  - master指令：指定仅由主线程执行的串行任务
  - task指令：任务并行

工作共享：FOR结构

Fortran	!$omp do [clause clause ...] *do-loops* !$omp end do

C/C++	#pragma omp for [clause clause ...]
	{
	for-loops
	}

- **只负责工作分享，不负责并行域的产生和管理，一般需放在并行域中**
- **如果不放在并行域内，则只能串行执行**
- **结尾处有隐式同步，可用的子句（clause）包括：**

private(*list*), **firstprivate**(*list*), **lastprivate**(*list*)
reduction(*op* : *list*)
schedule(*kind[, chunk_size]*)
ordered
nowait

工作共享：FOR结构

串行循环

- 将循环变量从初始值开始，逐次递增或递减，直至满足结束条件，其间对每个循环变量的取值都将执行一次循环体内的代码，且循环体内的代码是依次串行执行的

OpenMP并行循环

- 假定总共有N次循环，OpenMP对循环的任务分配就是将这N次循环进行划分，然后让每个并发线程各自负责其中的一部分循环工作，因此必须确保每次循环之间的数据的相互独立性

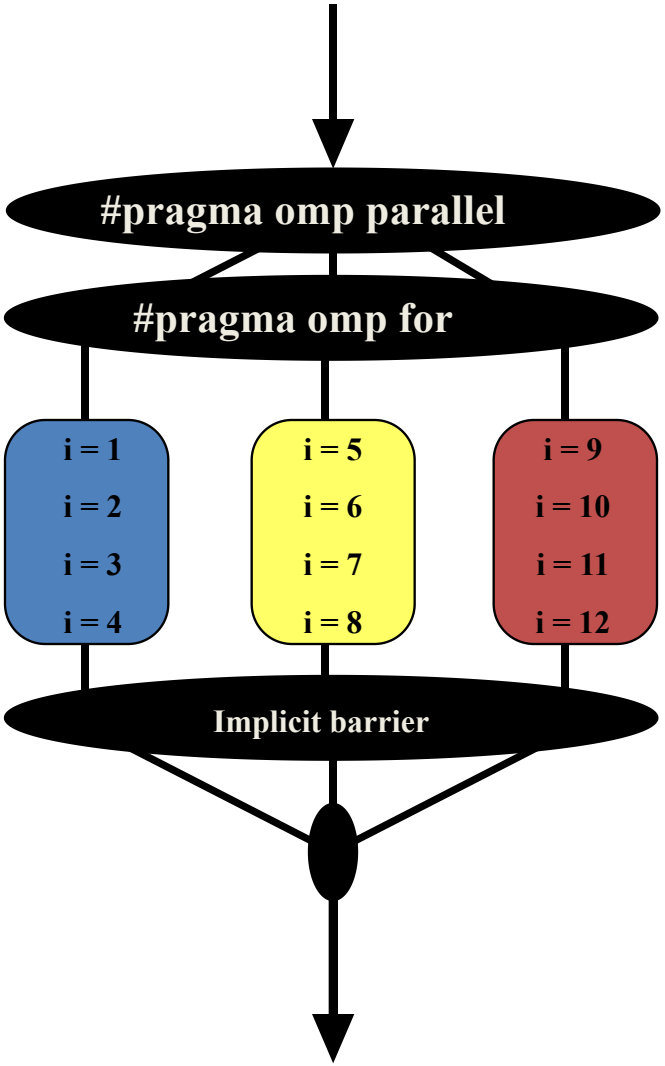
† 循环变量只能是整型或指针

† 将任务划分后分发给并发线程称为“调度”（schedule）

工作共享：FOR结构

```
#pragma omp parallel num_threads(4)
#pragma omp for
for(i = 1, i < 13, i++)
   c[i] = a[i] + b[i]
```

- 每个线程分配到一组迭代
- 线程在omp for语句结束后会等待其它线程



FOR举例

```
#include <omp.h>
#include <stdio.h>
#define N 10000
int main()
{
    int A[N], B[N], i;
    #pragma omp parallel num_threads(4)
    {
        #pragma omp for
        for(i=0; i<N; i++)
        {
            B[i]=i; A[i]=2*B[i];
        }
    }
    printf("A[n]=%d, B[n]=%d\n", A[N-1], B[N-1]);
    return 0;
}
```

for (循环变量赋初值; 循环条件; 循环变量增量)

循环体 // 循环体中不能修改循环变量的值

并行域与FOR合并

- **如果并行域中只有循环共享结构，则可以合写在一起**

Fortran	!$omp parallel do [clause clause ...] *do-loops* !$omp end parallel do

C/C++	#pragma omp parallel for [clause clause ...]
	{
	for-loops
	}

FOR举例

```
#define N 100000000
int main()
{
    static int A[N], B[N], i;  clock_t t0, t1;
    t0 = clock();
#pragma omp parallel for num_threads(4)
    for(i=0; i<N; i++){
       B[i]=i;
       A[i]=2*B[i];
    }
    t1 = clock();
    printf("A[n]=%d, B[n]=%d\n", A[N-1], B[N-1]);
    printf("Elapsed time: %.2e\n", (double)(t1-
t0)/CLOCKS_PER_SEC);
    return 0;
}
```

负载均衡问题

```
...
int main()
{
    ...
    sum = 0.0;
    #pragma omp parallel for num_threads(4)
    for(i = 0; i <= n; i++) {
        sum += f(i);
    }
    ...
}
double f(int i) {
    int j, start = i*(i+1)/2, finish = start + i;
    double return_val = 0.0;
    for(j = start; j <= finish; j++) {
        return_val += sin(j);
    }
    return return_val;
}
```

f函数调用所需要的时间，与参数i的大小成正比

SCHEDULE

- 在循环共享结构中，将任务划分后分发给各个线程称为调度(schedule)
- 任务调度的方式直接影响程序的效率：（1）任务的均衡程度；（2）循环体内数据访问顺序与相应的 cache 冲突情况。

循环体任务的调度基本原则

- 分解代价低：分解方法要快速，尽量减少分解任务而产生的额外开销
- 任务计算量要均衡
- 尽量避免高速缓存（cache）冲突，提高 cache 命中率

高速缓存cache

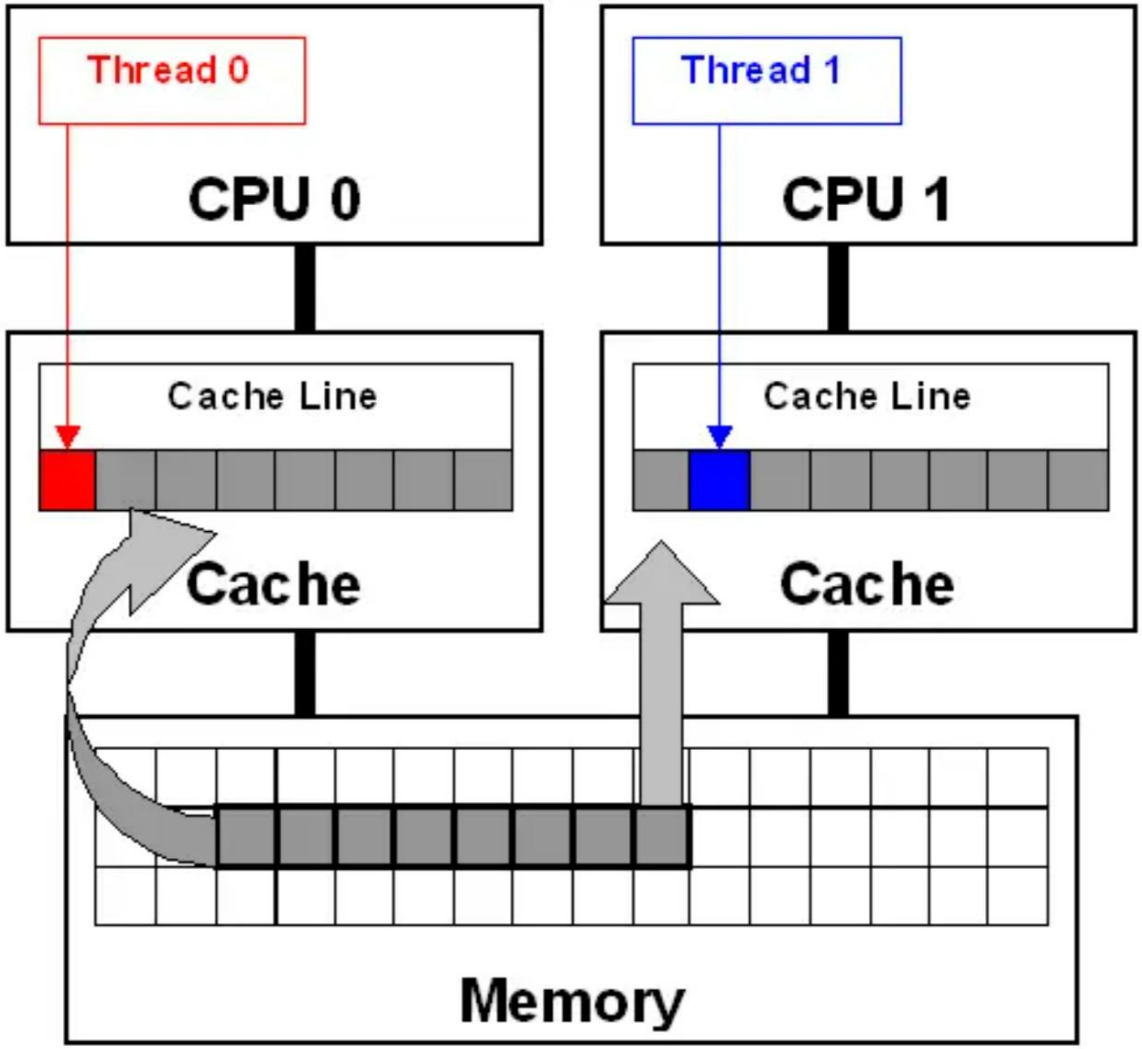
高速缓存（cache）的关键特性是以连续单元的数据块的形式组成的，当处理器需要引用某个数据块的一个或几个字节时，这个块的所有数据就会被传送到高速缓存中。因此，如果接下来需要引用这个块中的其他数据，则不必再从主存中调用它，这样就可以提高执行效率。

在多处理机系统中，不同的处理器可能需要同一个数据块的不同部分（不是相同的字节），尽管实际数据不共享（处理器有各自的高速缓存），但如果一个处理器对该块的其他部分写入，由于高速缓存的一致性协议，这个块在其他高速缓存上的拷贝就要全部进行更新或者使无效，这就是所谓的“假共享”，它对系统的性能有负面的影响。

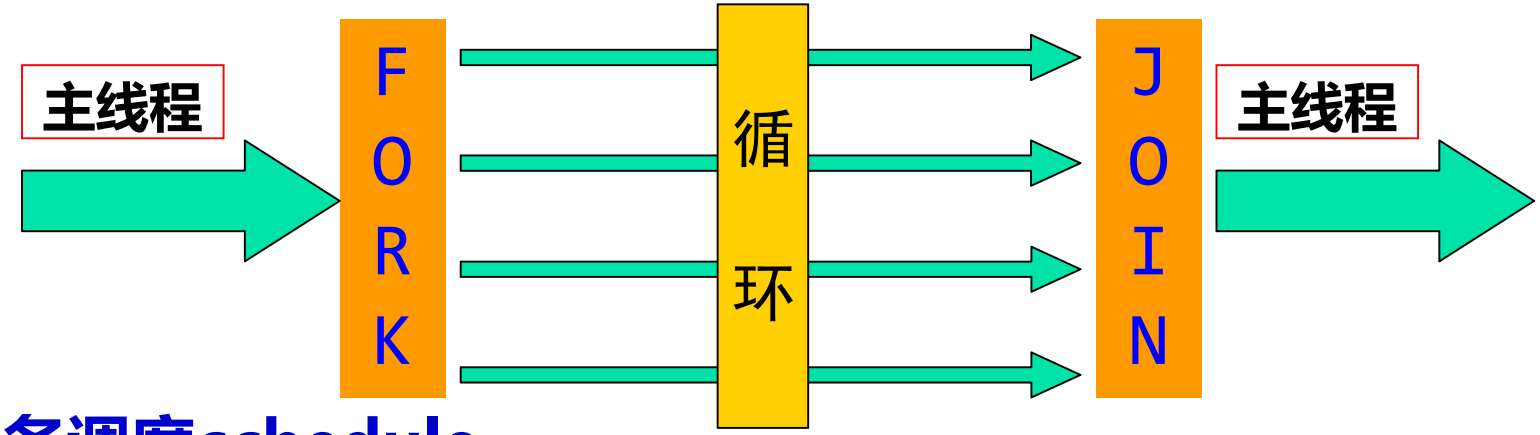
比如：两个处理器A和B访问同一个数据块的不同部分，如果处理器A修改了数据，则高速缓存一致协议将更新或者使处理器B 中的高速缓存块无效。而在此时处理器B可能也修改了数据，则高速缓存一致协议反 过来又要将处理器A 中的高速缓存块进行更新或者使无效。如此往复，就会导致高速缓存块的乒乓效应（ping-pong effect）。

伪共享问题

- 每个CPU对应的Cache把内存中同一段地址区域中（Memory中的灰色区域）的值拷贝过去，并且会保持这段地址中值的一致性
- 当CPU0中修改了这段区域的第一个值（红色所示），那么为了保持一致，会先通过某些机制，更新CPU 1的Cache中对应区域的值，然后才能执行其他任务
- CPU1修改了这段地址区域的第二个值（蓝色所示），同样为了保持一致，也会更新CPU0中的Cache中的内容
- 程序会来回地不停更新另一个Cache，导致程序执行变慢



SCHEDULE



- 任务调度schedule
  - schudule(static, chunk)
    静态分配，chunk 为任务块的大小，每个任务块被轮转分配给各线程
  - schudule(dynamic, chunk)
    动态分配，chunk 为任务块的大小，按先来先服务原则分配
  - schudule(guided, chunk)
    动态分配，任务块大小可变，先大后小，chunk 指定最小任务的大小
  - schudule(runtime)
    具体调度方式到运行时才进行，由环境变量 OMP_SCHEDULE 确定

SCHEDULE方式

schudule (static, chunk)

schudule (static)

- 循环任务被划分为chunk 大小的子任务，然后被轮转的分配给各个线程
- 省略chunk，则循环任务被划分成（近似）相同大小的子任务，每个线程被分配一个子任务

例： 假如线程数为 4，总任务量为 40，则

schudule(static)

T0	T1	T2	T3

schudule(static, 4)

T0	T1	T2	T3	T0	T1	T2	T3	T0	T1

SCHEDULE方式

schudule(dynamic, chunk)

schudule(dynamic)

- 基于先来先服务方式分配给各线程
- 当省略chunk 时，默认值为 1

schudule(guided, chunk)
schudule(guided)

- 类似dynamic，但任务块开始较大，然后变小，划分方式取决于编译器
- chunk 指定最小任务的大小，省略时默认值为1

schudule(runtime)

- 调度延迟到运行时，调度方式取决于环境变量OMP_SCHEDULE 的值

SCHEDULE方式

- 调度策略适用场景

调度策略	适用场景
STATIC	每次迭代的工作是可预测的并且相似的
DYNAMIC	每次迭代的工作是不可预测的，高度不规则的
GUIDED	Dynamic调度策略的一种特例，能够减少dynamic调度策略的负载

估计π值-串行

$$\pi = 4\left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \cdots + (-1)^n \frac{1}{2n+1} + \cdots\right)$$

```
int  n = 100000;
double factor = 1.0;
double sum = 0.0;

for(k = 0; k < n; k++) {
      sum += factor/(2 * k + 1);
      factor = -factor;
}
pi = 4.0 * sum;
```

估计π值-OpenMP版本1

```
int  n = 100000;
double factor = 1.0;
double sum = 0.0;

#pragma omp parallel for num_threads(thread_count) \
   reduction(+:sum)
for(k = 0; k < n; k++) {
      sum += factor/(2 * k + 1);
      factor = -factor;
}
pi = 4.0 * sum;
```

循环依赖或数据依赖问题

估计π值-OpenMP版本2

```
int  n = 100000;
double factor = 1.0;
double sum = 0.0;

#pragma omp parallel for num_threads(thread_count) \
   reduction(+:sum)
for(k = 0; k < n; k++) {
      factor=(k%2==0)?1.0:-1.0
      sum+=factor/(2*k+1);
}
pi = 4.0 * sum;
```

共享变量问题

估计π值-OpenMP版本3

```
int  n = 100000;
double factor = 1.0;
double sum = 0.0;

#pragma omp parallel for num_threads(thread_count) \
   reduction(+:sum) private(factor)
for(k = 0; k < n; k++) {
      factor=(k%2==0)?1.0:-1.0
      sum+=factor/(2*k+1);
}
pi = 4.0 * sum;
```

估计π值-OpenMP版本4

```
int  n = 100000;
double factor = 1.0;
double sum = 0.0;

#pragma omp parallel for num_threads(thread_count) \
   default(none) reduction(+:sum) private(k,factor) shared(n)
for(k = 0; k < n; k++) {
      factor=(k%2==0)?1.0:-1.0
      sum+=factor/(2*k+1);
}
pi = 4.0 * sum;
```

矩阵-向量乘法

- 如果A=(a_{ij})是一个m*n的矩阵，x是一个n维列向量，矩阵-向量的乘积Ax=y是个m维的列向量

$$
\begin{bmatrix}
a_{00} & a_{01} & \cdots & a_{0,n-1} \\
a_{10} & a_{11} & \cdots & a_{1,n-1} \\
\vdots & \vdots & & \vdots \\
a_{i0} & a_{i1} & \cdots & a_{i,n-1} \\
\vdots & \vdots & & \vdots \\
a_{m-1,0} & a_{m-1,1} & \cdots & a_{m-1,n-1}
\end{bmatrix}
\begin{bmatrix}
x_0 \\ x_1 \\ \vdots \\ x_{n-1}
\end{bmatrix}
=
\begin{bmatrix}
y_0 \\ y_1 \\ \vdots \\ y_i = a_{i0}x_0 + a_{i1}x_1 + \cdots a_{i,n-1}x_{n-1} \\ \vdots \\ y_{m-1}
\end{bmatrix}
$$

串行伪代码

```
/*For each row of A*/
for(i = 0; i < m; i++) {
    y[i] = 0.0;
    /*For each element of the row and each element of x*/
    for(j = 0; j < n; j++)
        y[i] += A[i][j] * x[j];
}
```

$$y_i = \sum_{j=0}^{n-1} a_{ij} x_j$$

矩阵-向量乘法 OpenMP版本

```
/* Spawn a parallel region explicitly scoping all variables */
#pragma omp parallel shared(A,x,y,chunk) private(i,j)
{
   #pragma omp for schedule (static, chunk)
   for(i = 0; i < m; i++) {
        y[i] = 0.0;
       /*For each element of the row and each element of x*/
        for(j = 0; j < n; j++)
           y[i] += A[i][j] * x[j];
   }
}
```

工作共享：SECTIONS结构

Fortran	!$omp sections [clause clause …] !$omp section *structured-block* !$omp section *structured-block* !$omp end sections [nowait]

C/C++	#pragma omp sections [clause clause …]
	{
	#pragma omp section
	structured-block
	#pragma omp section
	structured-block
	}

- sections 也可以与 parallel 合并，即 #pragma omp parallel sections

SECTIONS结构

- 指令sections创建一个工作共享域
- 域中的子任务由指令section创建，必须是一个相对独立的完整代码块
- 每个子任务都将只被一个线程执行
- 结束处隐含障碍同步，除非显式指明nowait
- sections可用的子句有

private(*list*)

firstprivate(*list*)

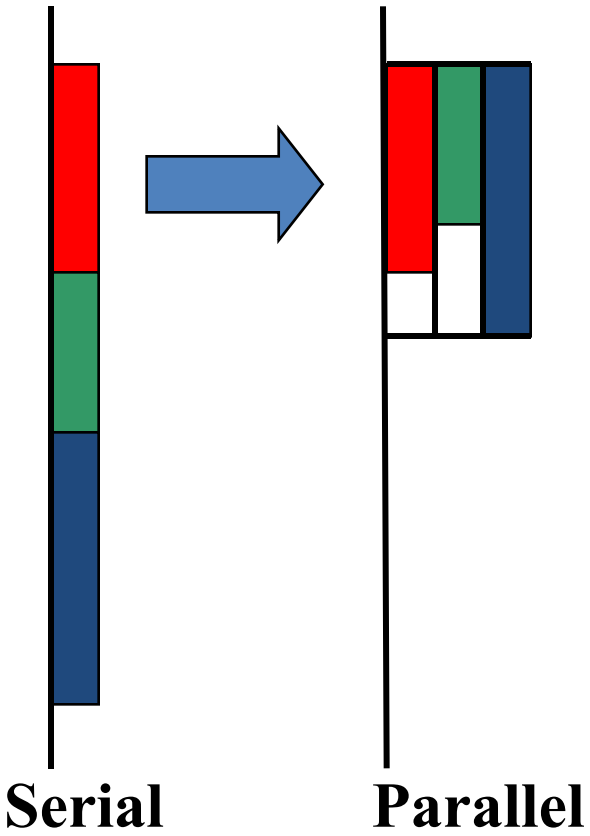
lastprivate(*list*)

reduction(*op* : *list*)

nowait

SECTIONS结构

```
#pragma omp parallel sections
{
    #pragma omp section
    phase1();
    #pragma omp section
    phase2();
    #pragma omp section
    phase3();
}
```



- 注意 sections 和 section 的区别
- #pragma omp section 必须在 sections 域中
- 共享域中的每个子任务必须由 section 指令创建
- 第一个子任务前面的 section 指令可以省略
- 子任务个数小于线程个数时，多余的线程空闲等待
- 子任务个数大于线程个数时，任务分配由编译器指定， 尽量负载平衡

工作共享： SINGLE结构

Fortran	!$omp single [clause clause …] *structured-block* !$omp end single [nowait copyprivate(*list*)]

C/C++	#pragma omp single [clause clause …]
	{
	structured-block
	}

- 用在并行域中，指定代码块只能由一个线程执行（不一定是主线程）
- 第一个遇到 single 指令的线程执行相应的代码，其它线程则在single结尾处等待，除非显式指明nowait
- 可用的子句有

private(*list*)
firstprivate(*list*)
copyprivate(*list*)
nowait

SINGLE示例

```
#pragma omp parallel private (tmp)
{
      do_many_things();
      #pragma omp single
      {
          exchange_boundaries();
      }
      do_many_other_things();
}
```

工作共享： MASTER结构

Fortran	!$omp master *structured-block* !$omp end master

C/C++	#pragma omp master
	{
	structured-block
	}

- master 块仅由线程组中的主线程执行
- 其它线程跳过并继续执行下面的代码， 即结尾处没有隐式同步
- 通常用于 I/O
- 与 single [nowait] 的区别：
  master 指定由主线程执行，而 single 由最先到达的线程执行

MASTER示例

```
#pragma omp parallel private (tmp)
{
    do_many_things();
    #pragma omp master
    {
        exchange_boundaries();
    }
    #pragma barrier
    do_many_other_things();
}
```

工作共享： TASK结构

Fortran	!$omp task [clause clause ...] *structured-block* !$omp end task

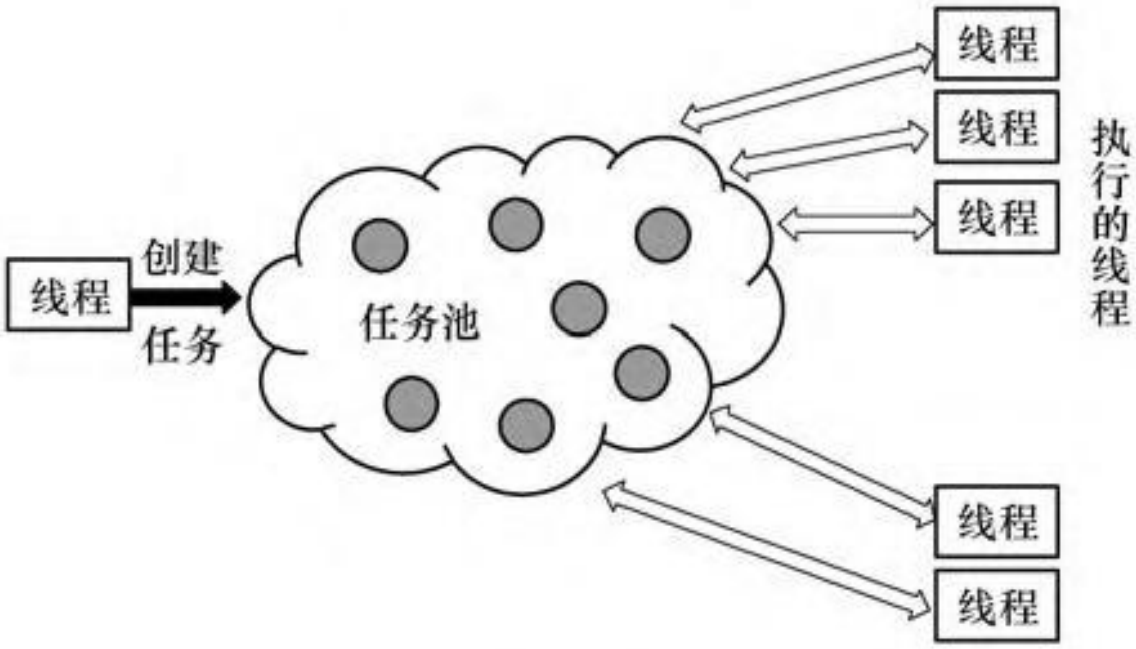
C/C++	#pragma omp task [clause clause ...]
	{
	structured-block
	}

- task定义了一个任务，该任务可能被当前线程立即执行，也可能被加入任务队列中等待条件满足时由线程组中某线程执行
- 任务并行比数据并行粒度更大，且更加灵活，能够处理递归等任务数量不确定的情况

if (*scalar-expression*)
final(*scalar-expression*)
default(shared | none)
private(*list*), firstprivate(*list*)
shared(*list*)
untied
mergeable

工作共享： TASK结构

- 指令task主要适用于不规则的循环迭代（如do-while）和递归的函数调用， 这些都是无法利用指令for完成的情况



工作共享： TASK结构

```
#pragma omp parallel
{
  #pragma omp single
  {
    node * p = head;
    while (p) {
    #pragma omp task firstprivate(p)
      process(p);
    p = p->next;
    }
  }
}
```

1. Create a team of threads.

2. One thread executes the **single** construct

… other threads wait at the implied barrier at the end of the single construct

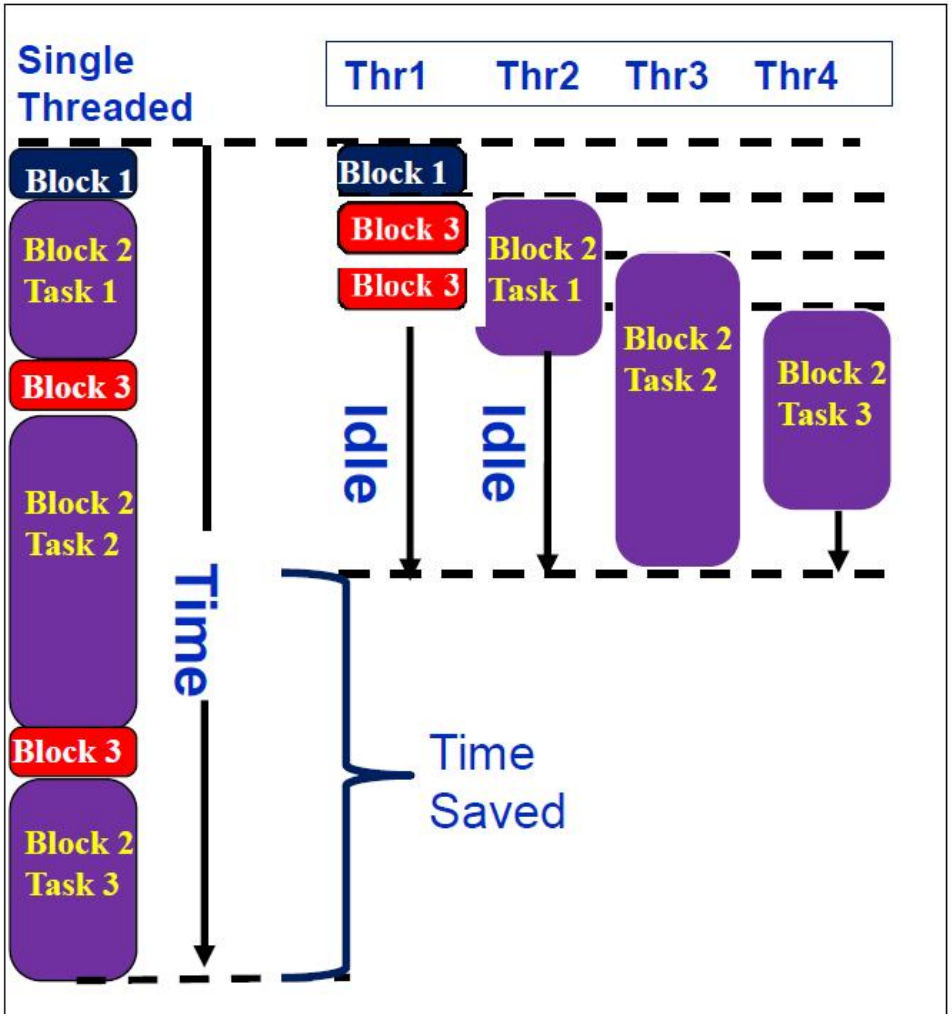
3. The “single” thread creates a task with its own value for the pointer p

4. Threads waiting at the barrier execute tasks.

Execution moves beyond the barrier once all the tasks are complete

工作共享： TASK结构

```
#pragma omp parallel
{
  #pragma omp single
  {   //block 1
    node * p = head;
    while (p) { // block 2
    #pragma omp task
      process(p);
    p = p->next;   //block 3
    }
  }
}
```



变量的作用域

- 变量作用域的属性可以通过如下方式修改
  - default (shared | none)
- 作用域属性语句
  - shared(varname,…)
  - private(varname,…)

变量的作用域

- 私有变量在每个线程中都有一份存储
  - 私有变量在被创建时是未初始化状态
  - 即使私有变量在进入并行代码块之前已经被初始化，但是在进入并行代码块时仍然是未初始化的

```
void* work(float* c, int N) {
    float x, y;
    int i;
    x = y = 100;
    #pragma omp parallel for private(x,y)
    for(i=0; i<N; i++) {
        x = a[i];
        y = b[i];
        c[i] = x + y;
    }
    printf("x = %d, y = %d\n", x, y);
}
```

变量的属性

- 如何决定哪些变量是共享哪些是私有?
  - 循环变量、临时变量、写变量一般应设置成私有的;
  - 数组变量、仅用于读的变量通常是共享的;
  - 能设置成共享的变量建议设置成共享的。
  - default(none)：所有变量必须显式指定是私有或共享

† 紧跟for 结构后面的循环变量默认是私有的，其他循环的循环变量需显式声明成私有的

数据共享属性子句

- 数据作用域属性子句 Data Sharing Attribute Clauses

private(list)	创建一个或多个变量的私有拷贝，即在每个线程中都创建一个同名局部变量，但没有初始值； 列表中的变量必须已定义，且不能是常量和引用； 列表中的多个变量用逗号隔开。
firstprivate(list)	private 的扩展， 创建私有拷贝的同时，将主线程中的同名变量的值作为初值。
lastprivate(list)	退出并行域时，将指定的私有拷贝的“最后”值复制到主线程中的同名变量中； “最后”：循环的最后一次迭代（按串行方式），或 sections 的最后一个 section（代码中）； 可能会增加额外开销，一般不建议使用，可以用共享变量等方式实现。

数据共享属性子句

- 数据作用域属性子句 Data Sharing Attribute Clauses

shared(list)	指定一个或多个变量为共享变量，即所有线程都 可以访问这些变量
default(...)	指定并行域内的变量的缺省属性，C语言支持 shared 和 none

数据共享属性子句

- 数据作用域属性子句 Data Sharing Attribute Clauses

copyin(list)	配合 threadprivate，用主线程同名变量的值对 threadprivate 的私有拷贝进行初始化
copyprivate(list)	配合 single，将 single 块中串行计算得到的变量 值广播到并行域中其它线程的同名变量中
reduction(op:list)	创建一个或多个变量的私有拷贝，在并行结束后对这些变量执行指定的归约操作（如求和），并将结果返回给主线程中的同名变量

REDUCTION子句

- 规约操作 reduction

```
void main (){
    int i;
    double ZZ, sum=0.0;
    omp_set_num_threads(NUM_THREADS)
    #pragma omp parallel for reduction(+:sum) private(ZZ)
    for (i=0; i< 1000; i++){
            ZZ = func(i);
            sum = sum + ZZ;
    }
}
```

reduction (op : list)
“list”中的变量在reduction语句所处的并行代码块中必须是共享的

- 在reduction 子句中，编译器为每个线程创建变量sum 的私有拷贝
- 退出并行域时，将这些值加在一起并把结果加到原始变量sum 中
- reduction 中的op 操作可以是：+, -, *, &&, ||, &, |, ^ 和内置函数 max, min

REDUCTION

- 规约操作时私有变量的初始值

+	0
*	1
-	0
&&	1
|| &	0
|	~0 0
^	0
max	相应变量类型的最小值
min	相应变量类型的最大值

OpenMP子句

- 其它字句

if(log_expr)	条件并行，满足指定条件时才执行相关操作
num_threads(int)	指定并行域内线程的个数
nowait	忽略并行线程或其它制导指令中暗含的障碍同步，使用时需小心
schedule(type,chunk)	指定循环任务的分配规则
collapse(int)	将多重循环转换为一层循环，然后进行任务划分

COLLAPSE子句

```
for(int i = 0; i < 5; i++){
    for(int j = 0; j < num; j++){
        y[i][j] = …;//computing
    }
}
```

```
#pragma omp parallel for collapse(2)
for(int i = 0; i < 5; i++){
    for(int j = 0; j < num; j++){
        y[i][j] = …;//computing
    }
}
```

改善负载均衡性，增加每个线程的计算量

同步

■ 同步结构 Synchronization Constructs

critical	避免线程竞争，其包含的代码同一时刻只能有一个线程执行
barrier	障碍同步：用在并行域内，所有线程执行到barrier都要停下等待，直到所有线程都执行到 barrier，然后再继续往下执行
atomic	确保一个特殊存储单元只能原子更新，即不允许多线程同时去写，只能用于单一赋值语句等特殊情况
flush	确保线程存储的临时视图与共享存储中的数据一致，并且保证一个变量在共享存储中的读/写顺序
ordered	指定并行域的循环按迭代顺序执行
taskwait	可配合 task 结构使用，创建任务调度点

同步：CRITICAL结构

Fortran	!$omp critical [(name)] *structured-block* !$omp end critical [(name)]

C/C++	#pragma omp critical [(name)]
	{
	structured-block
	}

- critical 块（临界区）限定同一时间只能有一个线程执行
- 所有线程将依次执行 critical 块
- 主要用于共享变量的更新，写文件等，避免数据竞争
- 并行域中可以包含多个critical块
- 线程到达临界区入口时等待，直到没有其它线程执行同名 critical 块
- 可以给每个critical块起名字
- 同名的临界区被看作是一个整体：同一时间，同名块中只能有一个线程
- 所有没有名字的 critical 块被看作是一个整体

CRITICAL示例

```
#pragma omp parallel num_threads(4) private(i,tid,mysum)
shared(a,h,mypi)
{
    tid = omp_get_thread_num();

    #pragma omp for
    for(i=1; i<n; i++)
    {
        mysum= mysum + f(a+i*h);
    }
    #pragma omp critical
    mypi = mypi + mysum;
}
mypi = mypi + (f(a) + f(b))/2;
mypi = h*mypi;
```

同步：BARRIER结构

Fortran	!$omp barrier
C/C++	#pragma omp barrier

- 路障同步： 线程遇到该指令都停下来等待，直到同组所有线程都到达该点，然后再继续执行后面的代码
- barrier 指令必须放在同组所有线程都能到达或都不能到达的地方，否则可能会引起死锁！比如不能放在 single ， critical ， master ， section 中！在循环或选择结构中使用时也需要注意， 如：

```
if (myid<5) {
   #pragma omp barrier
}
```

BARRIER示例

- Barrier: Each thread waits until all threads arrive.

```
#pragma omp parallel shared (A, B, C) private(id)
{
        id=omp_get_thread_num();
        A[id] = big_calc1(id);
#pragma omp barrier
#pragma omp for
        for(i=0;i<N;i++){C[i]=big_calc3(I,A);}
#pragma omp for nowait
        for(i=0;i<N;i++){ B[i]=big_calc2(C,  i); }
        A[id] = big_calc3(id);
}
```

implicit barrier at the end of a for work-sharing construct

no implicit barrier due to nowait

implicit barrier at the end of a parallel region

同步：ATOMIC结构

Fortran	!$omp atomic [clause]
C/C++	#pragma omp atomic [clause]

- atomic指令保证对某个存储地址做原子改写， 不允许多个线程同时改写
- 该指令仅作用于紧随其后的一条语句， 所支持的运算参见OpenMP手册
  支持的子句：read | write | update | capture

- 使用 atomic 和 critical 都可以避免数据竞争
- 使用 atomic 可能比 critical 更高效：
  使用 atomic 指令可以并行地更新数组内的不同元素
  而使用critical的话，则只能串行执行

原子改写的含义是指：读取该存储地址的内容、做所需运算、然后把新值写回该存储地址这一连串操作不会被其它线程间断， 它保证所有操作要么全部完成，要么保持原封不动。

ATOMIC示例

```
#pragma omp parallel for shared(x, y, index, n)
for (i = 0; i < n; i++) {
    #pragma omp atomic
    x[index[i]] += work1(i);
    y[i] += work2(i);
}
```

atomic语句是critical语句的一个特例
只应用在对某个内存地址进行简单的更新操作，可以并行地更新数组内的不同元素

同步：FLUSH结构

Fortran	!$omp flush [(*list*)]
C/C++	#pragma omp flush [(*list*)]

- flush 标识数据同步点，在这些点上，系统将提供一致的内存视图，在该指令出现的点上，共享变量被写回内存
- 不带 list，则表示针对所有变量
- 下列指令隐含不带 list 的 flush 指令：

  barrier，critical，parallel，ordered

注：若有nowait 子句，则不再隐含flush 指令

同步：ORDERD结构

Fortran	!$omp ordered *structured-block* !$omp end ordered

C/C++	#pragma omp ordered
	{
	structured-block
	}

- 指定循环必须按串行时的顺序执行
- ordered 只能出现在 for 或 parallel for 指定的循环区
- 一个循环体中最多只能有一个 ordered 块

ORDERD示例

```
#include <stdio.h>
#include <omp.h>

int main()
{
  const int niter = 10;
  #pragma omp parallel for ordered      //
  for (int i = 0; i < niter; i++) {
    int thr = omp_get_thread_num();
    printf("unordered iter %d of %d on thread %d\n", i, niter, thr);
  #pragma omp ordered              // 这里是需要顺序执行的部分
    printf("ordered iter %d of %d on thread %d\n", i, niter, thr);
  }
  return 0;
}
```

```
unordered iter 0 of 10 on thread 0
ordered iter 0 of 10 on thread 0
unordered iter 1 of 10 on thread 0
ordered iter 1 of 10 on thread 0
unordered iter 2 of 10 on thread 0
unordered iter 8 of 10 on thread 3
unordered iter 3 of 10 on thread 1
ordered iter 2 of 10 on thread 0
ordered iter 3 of 10 on thread 1
unordered iter 4 of 10 on thread 1
ordered iter 4 of 10 on thread 1
unordered iter 5 of 10 on thread 1
ordered iter 5 of 10 on thread 1
unordered iter 6 of 10 on thread 2
ordered iter 6 of 10 on thread 2
unordered iter 7 of 10 on thread 2
ordered iter 7 of 10 on thread 2
ordered iter 8 of 10 on thread 3
unordered iter 9 of 10 on thread 3
ordered iter 9 of 10 on thread 3
```

数据环境

■ 数据环境指令 Data Environment Constructs

threadprivate(list)	将一个或多个私有变量声明为全局的，即在多个并行域中使用时，保留私有变量在上次并行域中的值； 可以与copyin子句联合使用，将主线程的值广播给其他线程。

Fortran	!$omp threadprivate(*list*)
C/C++	#pragma omp threadprivate(*list*)

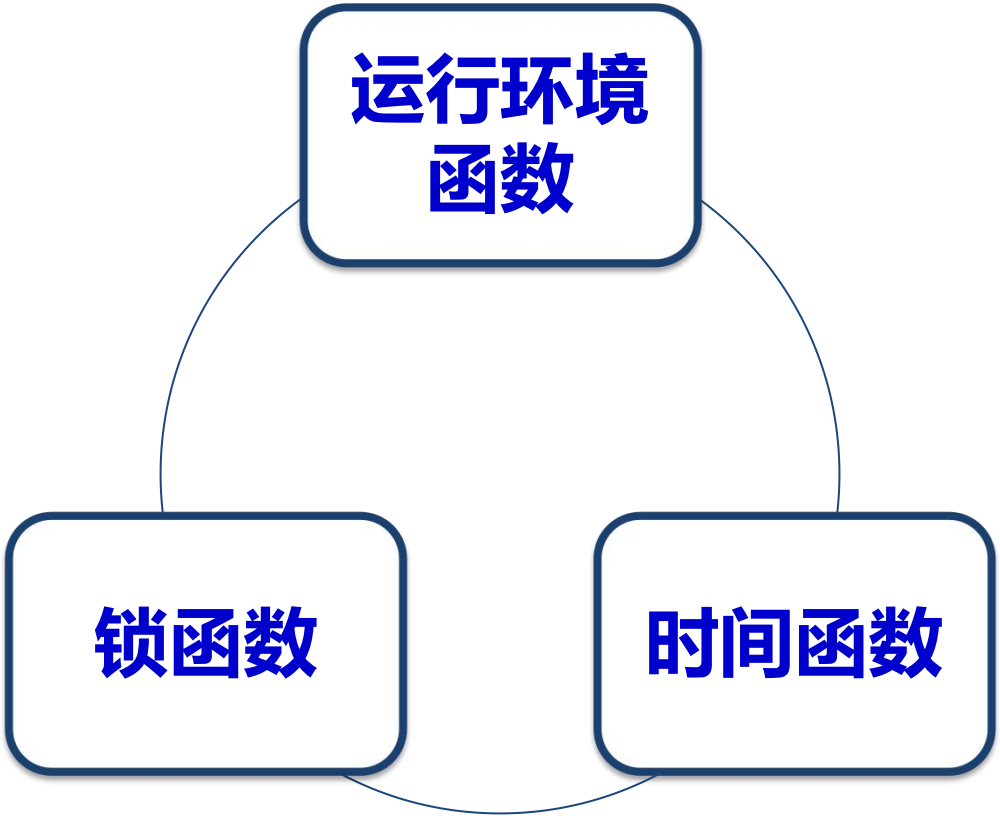
- 指定变量或公共数据块是线程私有的，且在同一个线程内是全局的（多个并行域）
- threadprivate 需出现在变量声明之后，其它语句之前
- 数据在同一个线程内是全局的

THREADPRIVATE示例

```
int a;
#pragma omp threadprivate(a)
#pragma omp parallel // 第一个并行域
{
    a=omp_get_thread_num();
}

. . .    . . .
#pragma omp parallel // 第二个并行域
{
    . . . . . .
}
```

- 变量 a 在第一个并行域中获取不同的值
- 在第二个并行域内，a保持原有的值

要素2：运行时库函数



运行环境函数

omp_set_num_threads(int)	设置并行域中的线程个数（用在串行域中）
omp_get_num_threads()	返回当前并行域中的线程个数
omp_get_max_threads()	返回并行域中缺省可用的最大线程个数
omp_get_thread_num()	返回当前线程的线程号，0号为主线程
omp_get_num_procs()	返回系统中处理器的个数
omp_in_parallel()	判断是否在并行域中
omp_set_dynamic(int)	启用或关闭线程数目动态改变功能（用在串行域中）
omp_get_dynamic()	判断系统是否支持动态改变线程数目
omp_set_nested(int)	启用或关闭并行域嵌套功能（缺省为关闭）
omp_get_nested()	判断系统是否支持并行域的嵌套

锁函数

什么是锁

- OpenMP提供了一些锁函数用于避免任务竞争，作用类似于atomic和 critical
- OpenMP锁是一个特殊的变量，称为锁变量
- 锁变量的取值有：uninitialized, unlocked, locked
- 如果一个锁处于 unlocked状态，则任务就可以设置该锁，并拥有这个锁，只有拥有该锁的任务才能解锁，其他任务只有等解锁后才能使用这个锁
- 锁变量有两类：简单锁和嵌套锁，后者的区别在于可以设置多次而不阻塞，分别是 omp_lock_t 和omp_nest_lock_t

锁函数

锁操作：初始化，销毁，上锁，解锁，测试

■锁函数一般按如下顺序进行调用

① 用 omp_init_lock/ omp_init_nest_lock初始化锁变量

② 调用 omp_set_lock/ omp_set_nest_lock上锁，是阻塞型函数

③ 线程须调用 omp_unset_lock/ omp_unset_nest_lock来解锁

④ 当不再需要该锁，调用 omp_destroy_lock/ omp_destroy_nest_lock 来释放锁资源，即设为为初始化

⑤ 函数 omp_test_lock/ omp_test_nest_lock尝试去上锁，非阻塞型，若上锁成功，返回 1（简单锁）或嵌套层数（嵌套锁），否则返回 0

锁函数

omp_init_lock(omp_lock_t * *lock*)	初始化一个简单锁
omp_destroy_lock(omp_lock_t * *lock*)	销毁一个简单锁
omp_set_lock(omp_lock_t * *lock*)	上锁操作
omp_unset_lock(omp_lock_t * *lock*)	解锁操作
omp_test_lock(omp_lock_t * *lock*)	非阻塞上锁操作

omp_init_nest_lock(omp_nest_lock_t * *lock*)	初始化一个嵌套锁
omp_destroy_nest_lock(omp_nest_lock_t * *lock*)	销毁一个嵌套锁
omp_set_nest_lock(omp_nest_lock_t * *lock*)	上锁操作
omp_unset_nest_lock(omp_nest_lock_t * *lock*)	解锁操作
omp_test_nest_lock(omp_nest_lock_t * *lock*)	非阻塞上锁操作

锁函数举例

```
int main()
{
int k;  omp_lock_t lock;

omp_init_lock(&lock); // 初始化锁
#pragma omp parallel num_threads(4) private(k)
{
  omp_set_lock(&lock); // 上锁
  for(k=0; k<4; k++)
    printf("myid=%d, k=%d\n", omp_get_thread_num(), k);
  omp_unset_lock(&lock); // 解锁
}
omp_destroy_lock(&lock); // 释放锁
return 0;
}
```

```
myid=1, k=0
myid=1, k=1
myid=1, k=2
myid=1, k=3
myid=0, k=0
myid=0, k=1
myid=0, k=2
myid=0, k=3
myid=2, k=0
myid=2, k=1
myid=2, k=2
myid=2, k=3
myid=3, k=0
myid=3, k=1
myid=3, k=2
myid=3, k=3
```

时间函数

omp_get_wtime()	获取 wall time，以秒为单位，双精度型的实数
omp_get_wtick()	获取每个时钟周期的秒数，即 omp_get_wtime 的精度

- omp_get_wtime

returns elapsed wall clock time in seconds

Fortran	double precision function omp_get_wtime()
C/C++	double omp_get_wtime(void);

- omp_get_wtick

returns the number of seconds between successive clock ticks

Fortran	double precision function omp_get_wtick()
C/C++	double omp_get_wtick(void);

时间函数举例

C/C++

```
double t0, t1;
t0=omp_get_wtime();
... work to be timed ...
t1=omp_get_wtime();
printf("Work took %f seconds\n", t1-t0);
```

要素3：环境变量

OpenMP 提供了一环境变量来控制并行代码的执行

OMP_SCHEDULE	设置循环任务的调度模式
OMP_NUM_THREADS	设置线程个数
OMP_DYNAMIC	设置是否开启线程数的动态变化功能
OMP_PROC_BIND	设置线程是否与处理器绑定
OMP_NESTED	设置是否开启并行域的嵌套功能
OMP_STACKSIZE	线程的栈的大小，缺省单位是 K
OMP_WAIT_POLICY	设置线程等待时是否占用处理器资源
OMP_MAX_ACTIVE_LEVELS	设置并行域嵌套最大层数
OMP_THREAD_LIMIT	设置整个程序所能使用的最大线程数

环境变量

- 在 Linux 下修改环境变量的一般格式

```
export  环境变量名 = 值
```

```
echo $环境变量名   // 查看变量的值
```

```
export OMP_SCHEDULE=dynamic
export OMP_SCHEDULE="guided,4"
```

```
export OMP_NUM_THREADS=4
```

```
export OMP_PROC_BIND=true
export OMP_PROC_BIND=false
```

Reference

- OpenMP Application Programming Interface, https://www.openmp.org/wp-content/uploads/OpenMP-API-Specification-5-2.pdf
- Pthread, https://hpc-tutorials.llnl.gov/posix/
- Intel® oneAPI Threading Building Blocks, https://www.intel.com/content/www/us/en/developer/tools/oneapi/onetbb.html#gs.sxzdka
- Blaise Barney, Lawrence Livermore National Laboratory, Introduction to Parallel Computing, https://computing.llnl.gov/tutorials/parallel_comp/